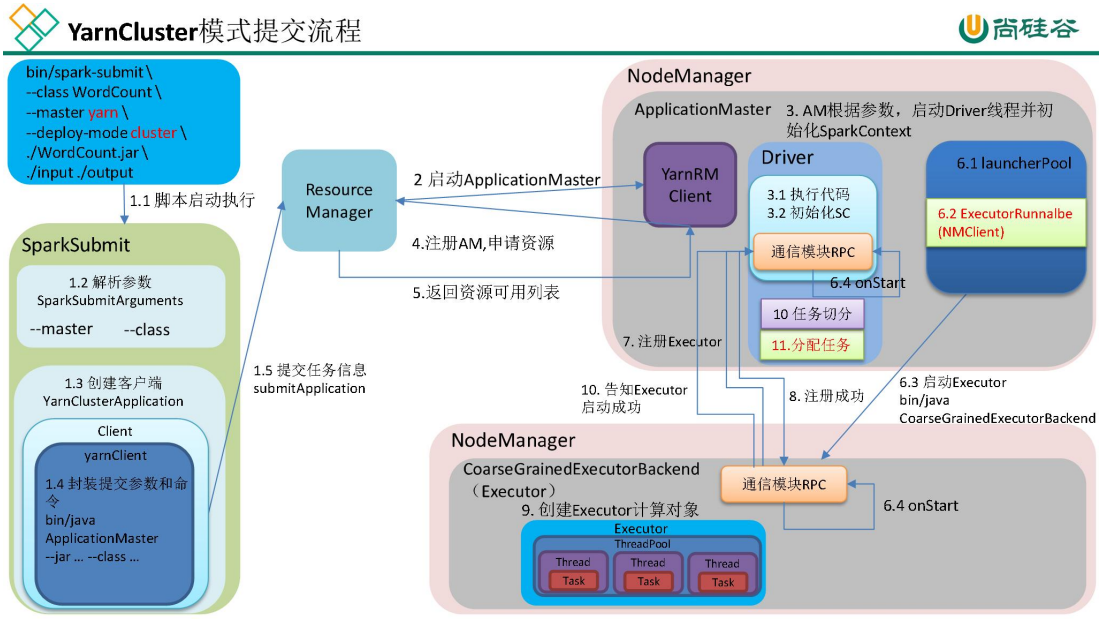
尚硅谷大数据技术之 Spark 内核

（作者：尚硅谷大数据研发部）

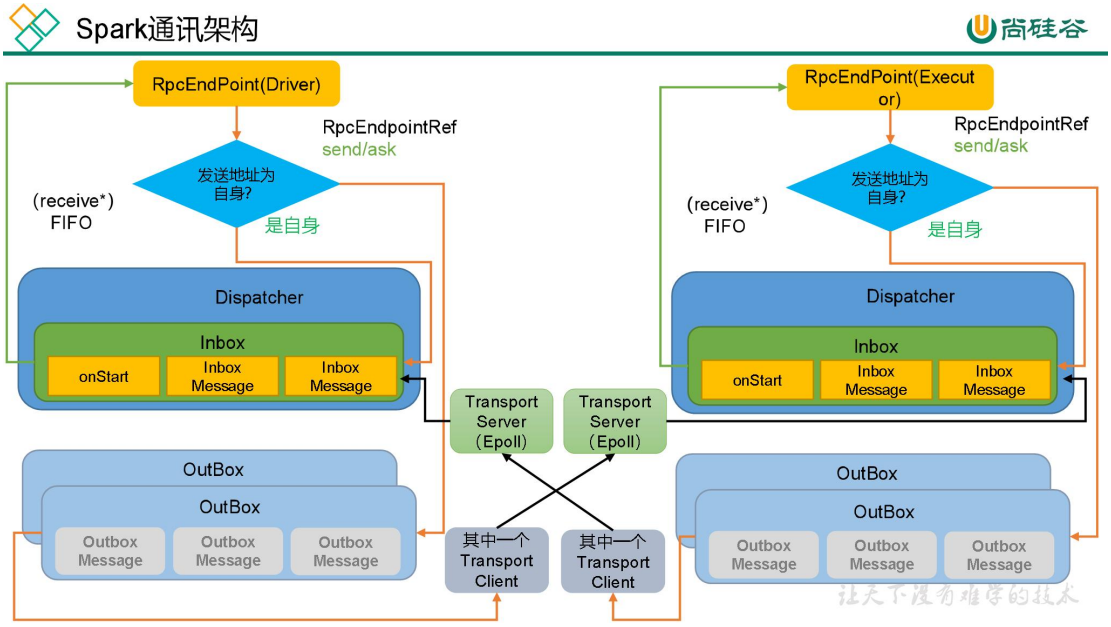
版本：V3.0

第 0 章 源码全流程

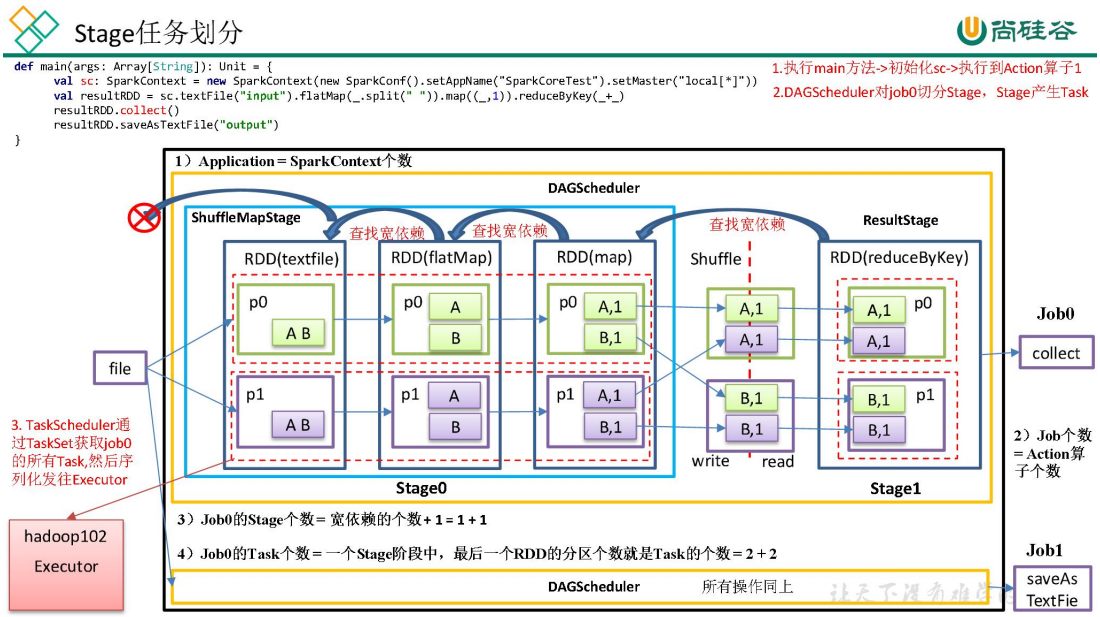
0.1 Spark 提交流程（YarnCluster）



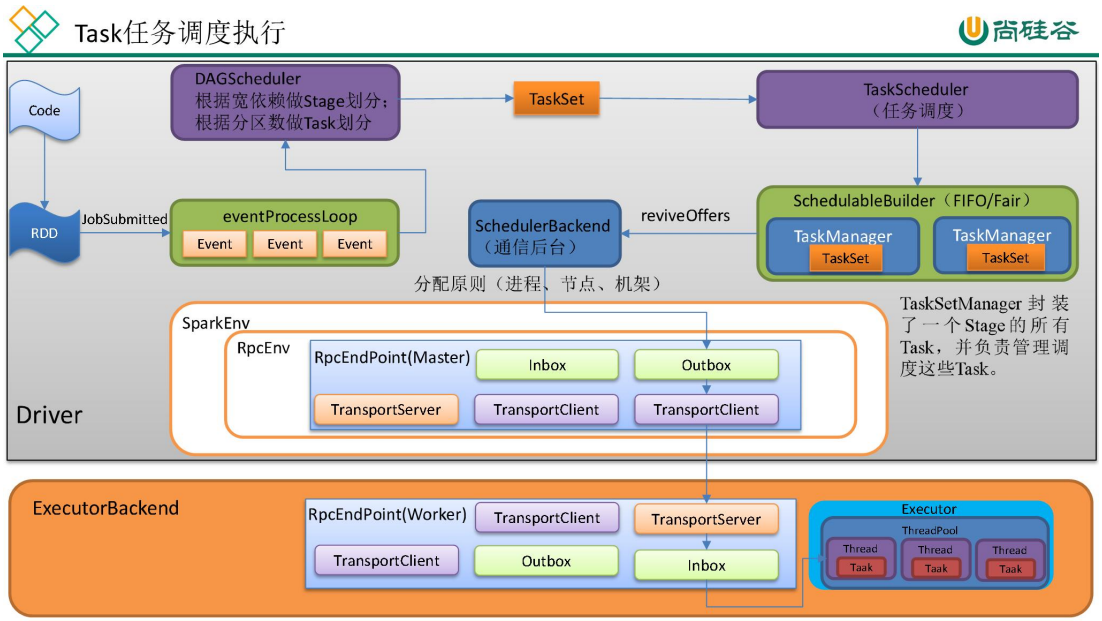
0.2 Spark 通讯架构



0.3 Spark 任务划分



0.4 Task 任务调度



Task FIFO调度算法

```
val priority1 = s1.priority
val priority2 = s2.priority
var res = math.signum(priority1 - priority2)
if (res == 0) {
  val stageId1 = s1.stageId
  val stageId2 = s2.stageId
  res = math.signum(stageId1 - stageId2)
}
res < 0
```

- **priority**(优先级)：生成 TaskSet实例时，传入的JobID
- **stageId**：生成 TaskSet实例时，传入的stage的ID

```
taskScheduler.submitTasks(new TaskSet(
  tasks.toArray, stage.id,
stage.latestInfo.attemptNumber, jobId,
properties))
```

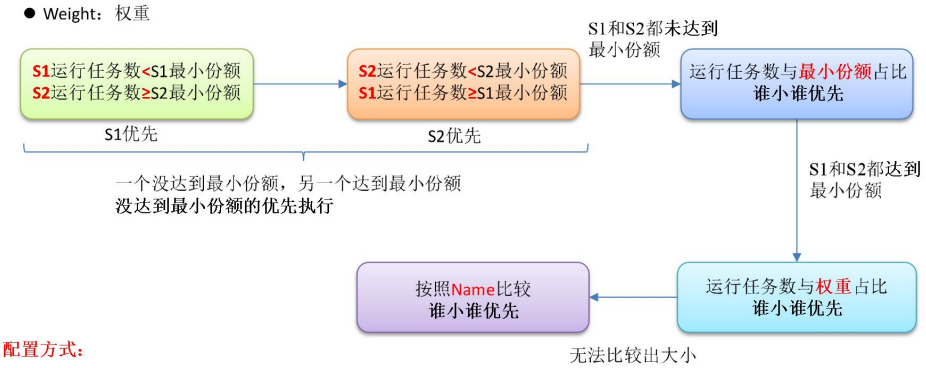
遇到一个行动算子生成一个Job，jobId是递增的

同一个job内部的stageId是递增的

- 先生成的Job (对应SPARK中代码就是先执行的ACTION)先进行调度
- 如果是同一个Job，最上层的STAGE先进行调度

Task公平调度算法

- runningTasks：运行中的任务数
- minShare：最小份额
- Weight：权重

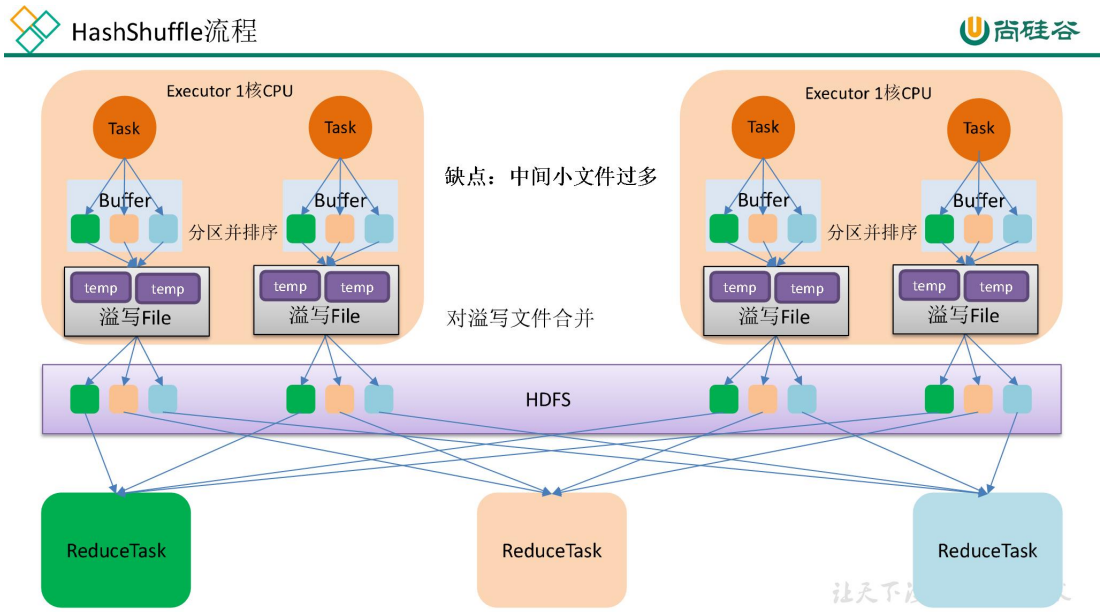


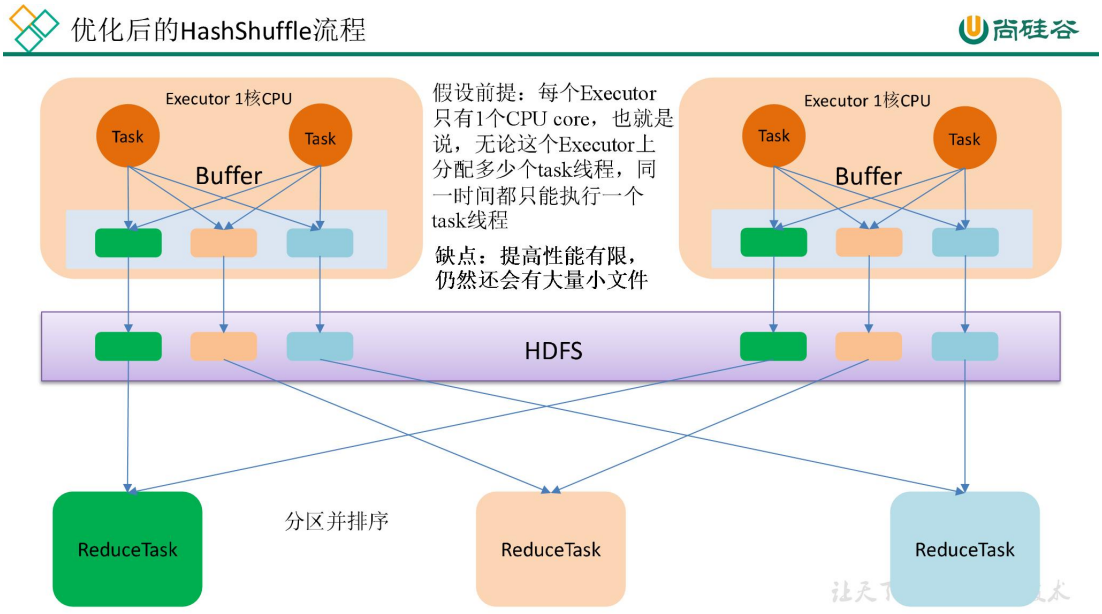
配置方式：

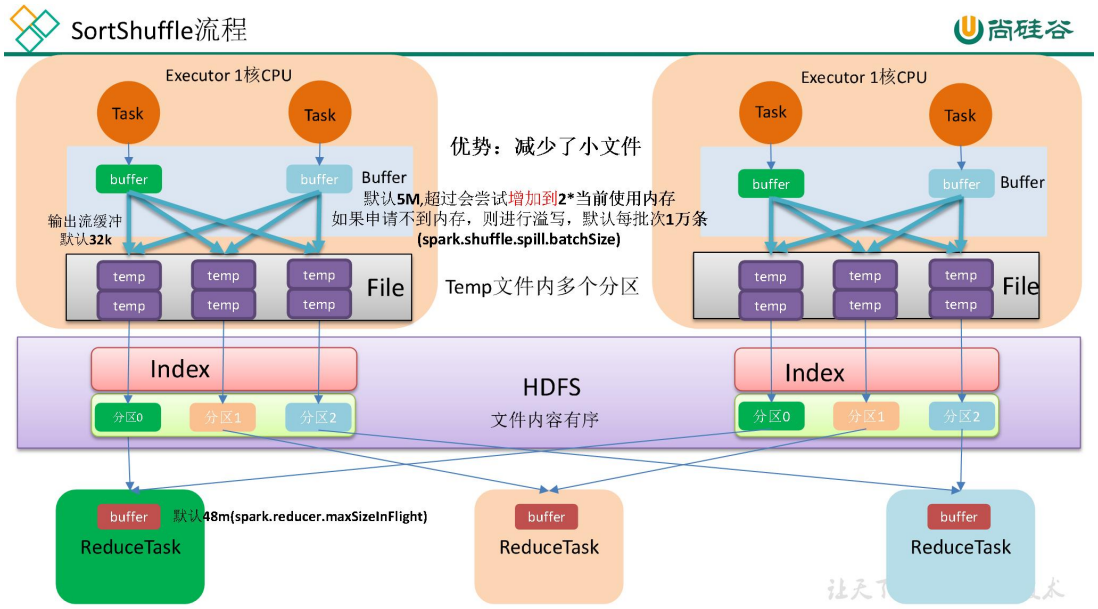
编辑 SPARK_HOME/conf/fairscheduler.xml

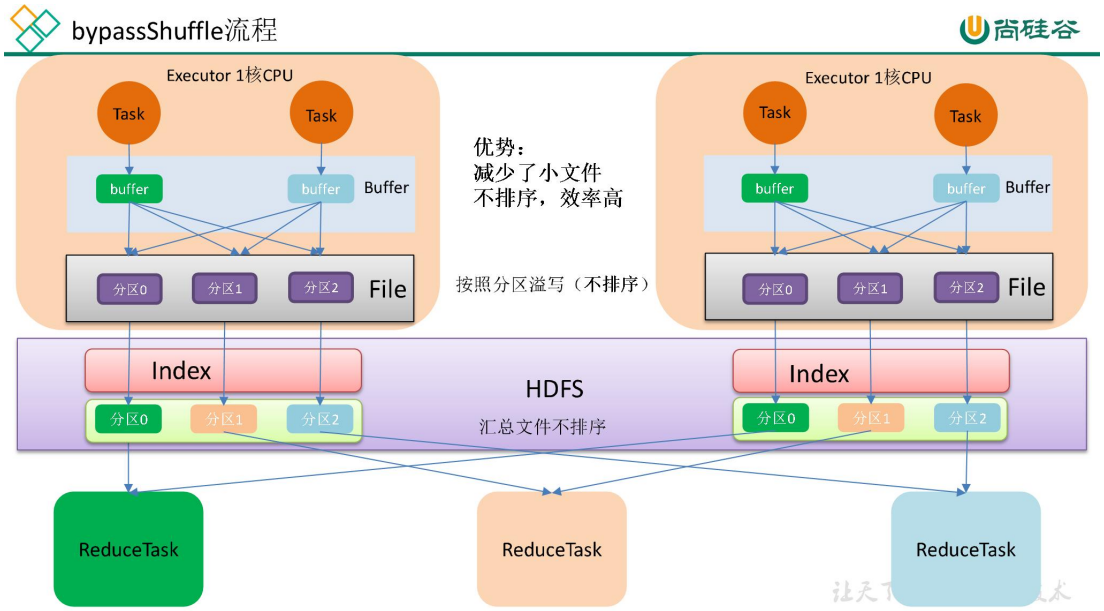
指定schedulingMode(FAIR)、weight、minShare

0.5 Shuffle 原理









第 1 章 环境准备及提交流程

1.1 程序起点

1）spark-3.0.0-bin-hadoop3.2\bin\spark-submit.cmd

=> cmd /V /E /C ""%~dp0spark-submit2.cmd" %*"

2）spark-submit2.cmd

=> set CLASS=org.apache.spark.deploy.SparkSubmit

"%~dp0spark-class2.cmd" %CLASS% %*

3）spark-class2.cmd

=> %SPARK_CMD%

4）在 spark-class2.cmd 文件中增加打印%SPARK_CMD%语句

```
echo %SPARK_CMD%
%SPARK_CMD%
```

5）在 spark-3.0.0-bin-hadoop3.2\bin 目录上执行 cmd 命令

6）进入命令行窗口，输入

```
spark-submit --class org.apache.spark.examples.SparkPi --master local[2] ../examples/jars/spark-examples_2.12-3.0.0.jar 10
```

```
E:\02_software\spark-3.0.0-bin-hadoop3.2\bin>spark-submit --class org.apache.spa
rk.examples.SparkPi --master local[2] ../examples/jars/spark-examples_2.12-3.0.0
.jar 10
Missing Python executable 'python', defaulting to 'E:\02_software\spark-3.0.0-bi
n-hadoop3.2\bin\..' for SPARK_HOME environment variable. Please install Python o
r specify the correct Python executable in PYSPARK_DRIVER_PYTHON or PYSPARK_PYTH
ON environment variable to detect SPARK_HOME safely.
E:\02_software\Java\jdk1.8.0_144\bin\java -cp "E:\02_software\spark-3.0.0-bin-ha
doop3.2\bin\..\conf\;E:\02_software\spark-3.0.0-bin-hadoop3.2\bin\..\jars\*" -Xm
x1g org.apache.spark.deploy.SparkSubmit --master local[2] --class org.apache.spa
rk.examples.SparkPi ../examples/jars/spark-examples_2.12-3.0.0.jar 10
```

7）发现底层执行的命令为

```
java -cp    org.apache.spark.deploy.SparkSubmit
```

说明：java -cp 和 -classpath 一样，是指定类运行所依赖其他类的路径。

8）执行 java -cp 就会开启 JVM 虚拟机，在虚拟机上开启 SparkSubmit 进程，然后开始执行 main 方法

java -cp =》开启 JVM 虚拟机 =》开启 Process（SparkSubmit）=》程序入口 SparkSubmit.main

9）在 IDEA 中全局查找（ctrl + n）：org.apache.spark.deploy.SparkSubmit，找到 SparkSubmit 的伴生对象，并找到 main 方法

```
override def main(args: Array[String]): Unit = {
    val submit = new SparkSubmit() {
    ... ...
    }
}
```

1.2 创建 Yarn Client 客户端并提交

1.2.1 程序入口

SparkSubmit.scala

```
override def main(args: Array[String]): Unit = {
    val submit = new SparkSubmit() {
    ... ...
        override def doSubmit(args: Array[String]): Unit = {
          super.doSubmit(args)
        }
    }
    submit.doSubmit(args)
```

```
}

def doSubmit(args: Array[String]): Unit = {
    val uninitLog = initializeLogIfNecessary(true, silent = true)
    // 解析参数
    val appArgs = parseArguments(args)
    … …
    appArgs.action match {
        // 提交作业
        case SparkSubmitAction.SUBMIT => submit(appArgs, uninitLog)
        case SparkSubmitAction.KILL => kill(appArgs)
        case SparkSubmitAction.REQUEST_STATUS => requestStatus(appArgs)
        case SparkSubmitAction.PRINT_VERSION => printVersion()
    }
}
```

1.2.2 解析输入参数

```
protected def parseArguments(args: Array[String]): SparkSubmitArguments = {
    new SparkSubmitArguments(args)
}
```

SparkSubmitArguments.scala

```
private[deploy] class SparkSubmitArguments(args: Seq[String], env: Map[String, String] =
sys.env)
    extends SparkSubmitArgumentsParser with Logging {
    ... ...
    parse(args.asJava)
    ... ...
}
```

SparkSubmitOptionParser.java

```
protected final void parse(List<String> args) {

    Pattern eqSeparatedOpt = Pattern.compile("(--[^=]+)=(.+)");

    int idx = 0;
    for (idx = 0; idx < args.size(); idx++) {
        String arg = args.get(idx);
        String value = null;

        Matcher m = eqSeparatedOpt.matcher(arg);
        if (m.matches()) {
            arg = m.group(1);
            value = m.group(2);
        }

        String name = findCliOption(arg, opts);
        if (name != null) {
            if (value == null) {
                … …
            }
            // handle 的实现类（ctrl + h）是 SparkSubmitArguments.scala 中
            if (!handle(name, value)) {
                break;
            }
```

```
                continue;
            }
            … …
        }
        handleExtraArgs(args.subList(idx, args.size()));
}
```

SparkSubmitArguments.scala

```
override protected def handle(opt: String, value: String): Boolean = {
        opt match {
            case NAME =>
                name = value
            // protected final String MASTER = "--master";    SparkSubmitOptionParser.java
            case MASTER =>
                master = value

            case CLASS =>
                mainClass = value
            ... ...
            case _ =>
                error(s"Unexpected argument '$opt'.")
        }
        action != SparkSubmitAction.PRINT  VERSION
}

private[deploy] class SparkSubmitArguments(args: Seq[String], env: Map[String, String] =
sys.env)
   extends SparkSubmitArgumentsParser with Logging {
        ... ...
        var action: SparkSubmitAction = null
        ... ...

        private def loadEnvironmentArguments(): Unit = {
            ... ...
            // Action should be SUBMIT unless otherwise specified
            // action 默认赋值 submit
            action = Option(action).getOrElse(SUBMIT)
        }
        ... ...
}
```

1.2.3 选择创建哪种类型的客户端

SparkSubmit.scala

```
private[spark] class SparkSubmit extends Logging {
        ... ...
        def doSubmit(args: Array[String]): Unit = {

            val uninitLog = initializeLogIfNecessary(true, silent = true)
            //  解析参数
            val appArgs = parseArguments(args)
            if (appArgs.verbose) {
               logInfo(appArgs.toString)
            }
            appArgs.action match {
```

```
                // 提交作业
                case SparkSubmitAction.SUBMIT => submit(appArgs, uninitLog)
                case SparkSubmitAction.KILL => kill(appArgs)
                case SparkSubmitAction.REQUEST_STATUS => requestStatus(appArgs)
                case SparkSubmitAction.PRINT_VERSION => printVersion()
            }
        }

        private def submit(args: SparkSubmitArguments, uninitLog: Boolean): Unit = {

            def doRunMain(): Unit = {
                if (args.proxyUser != null) {
                    … …
                } else {
                    runMain(args, uninitLog)
                }
            }

            if (args.isStandaloneCluster && args.useRest) {
                … …
            } else {
                doRunMain()
            }
        }

        private def runMain(args: SparkSubmitArguments, uninitLog: Boolean): Unit = {
            // 选择创建什么应用：YarnClusterApplication
            val (childArgs, childClasspath, sparkConf, childMainClass) =
prepareSubmitEnvironment(args)
            ... ...
            var mainClass: Class[_] = null

            try {
                mainClass = Utils.classForName(childMainClass)
            } catch {
                ... ...
            }
            // 反射创建应用：YarnClusterApplication
            val app: SparkApplication = if
(classOf[SparkApplication].isAssignableFrom(mainClass)) {
                mainClass.getConstructor().newInstance().asInstanceOf[SparkApplication]
            } else {
                new JavaMainApplication(mainClass)
            }
            ... ...
            try {
                //启动应用
                app.start(childArgs.toArray, sparkConf)
            } catch {
            case t: Throwable =>
                throw findCause(t)
            }
        }
        ... ...
}
```

SparkSubmit.scala

```
private[deploy] def prepareSubmitEnvironment(
        args: SparkSubmitArguments,
        conf: Option[HadoopConfiguration] = None)
        : (Seq[String], Seq[String], SparkConf, String) = {

    var childMainClass = ""
    ... ...
    // yarn 集群模式
    if (isYarnCluster) {
        // YARN_CLUSTER_SUBMIT_CLASS="org.apache.spark.deploy.yarn.YarnClusterApplication"
        childMainClass = YARN_CLUSTER_SUBMIT_CLASS
        ... ...
    }
    ... ...
    (childArgs, childClasspath, sparkConf, childMainClass)
}
```

1.2.4 Yarn 客户端参数解析

1）在 pom.xml 文件中添加依赖 spark-yarn

```
<dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-yarn_2.12</artifactId>
    <version>3.0.0</version>
</dependency>
```

2）在 IDEA 中全文查找（ctrl+n）org.apache.spark.deploy.yarn.YarnClusterApplication

3）Yarn 客户端参数解析

Client.scala

```
private[spark] class YarnClusterApplication extends SparkApplication {

  override def start(args: Array[String], conf: SparkConf): Unit = {
    ... ...
    new Client(new ClientArguments(args), conf, null).run()
  }
}
```

ClientArguments.scala

```
private[spark] class ClientArguments(args: Array[String]) {
    ... ...
    parseArgs(args.toList)

    private def parseArgs(inputArgs: List[String]): Unit = {
        var args = inputArgs
        while (!args.isEmpty) {
            args match {
                case ("--jar") :: value :: tail =>
                userJar = value
                args = tail

                case ("--class") :: value :: tail =>
                userClass = value
```

```
                    args = tail
                    ... ...
                    case _ =>
                    throw new IllegalArgumentException(getUsageMessage(args))
                }
            }
        }
        ... ...
}
```

1.2.5 创建 Yarn 客户端

Client.scala

```
private[spark] class Client(
        val args: ClientArguments,
        val sparkConf: SparkConf,
        val rpcEnv: RpcEnv)
        extends Logging {
        // 创建 yarnClient
        private val yarnClient = YarnClient.createYarnClient
        ... ...
}
```

YarnClient.java

```
public abstract class YarnClient extends AbstractService {

        @Public
        public static YarnClient createYarnClient() {
                YarnClient client = new YarnClientImpl();
                return client;
        }
        ... ...
}
```

YarnClientImpl.java

```
public class YarnClientImpl extends YarnClient {
        // yarnClient 主要用来和 RM 通信
        protected ApplicationClientProtocol rmClient;
        ... ...

        public YarnClientImpl() {
                super(YarnClientImpl.class.getName());
        }
        ... ...
}
```

1.2.6 Yarn 客户端创建并启动 ApplicationMaster

Client.scala

```
private[spark] class YarnClusterApplication extends SparkApplication {

  override def start(args: Array[String], conf: SparkConf): Unit = {
    // SparkSubmit would use yarn cache to distribute files & jars in yarn mode,
    // so remove them from sparkConf here for yarn mode.
    conf.remove(JARS)
```

```
        conf.remove(FILES)

        new Client(new ClientArguments(args), conf, null).run()
    }
}
```

```
private[spark] class Client(
        val args: ClientArguments,
        val sparkConf: SparkConf,
        val rpcEnv: RpcEnv)
        extends Logging {

        def run(): Unit = {
                this.appId = submitApplication()
                ... ...
        }

        def submitApplication(): ApplicationId = {
                var appId: ApplicationId = null
                try {
                        launcherBackend.connect()
                        yarnClient.init(hadoopConf)
                        yarnClient.start()

                        val newApp = yarnClient.createApplication()
                        val newAppResponse = newApp.getNewApplicationResponse()
                        appId = newAppResponse.getApplicationId()

                        ... ...
                        // 封装提交参数和命令
                        val containerContext = createContainerLaunchContext(newAppResponse)
                        val appContext = createApplicationSubmissionContext(newApp,
containerContext)

                        yarnClient.submitApplication(appContext)
                        ... ...
                        appId
                } catch {
                        ... ...
                }
        }
}

// 封装提交参数和命令
private def createContainerLaunchContext(newAppResponse: GetNewApplicationResponse)
        : ContainerLaunchContext = {
        ... ...
        val amClass =
                // 如果是集群模式启动 ApplicationMaster，如果是客户端模式启动
ExecutorLauncher
                if (isClusterMode) {

Utils.classForName("org.apache.spark.deploy.yarn.ApplicationMaster").getName
                } else {
```

```
Utils.classForName("org.apache.spark.deploy.yarn.ExecutorLauncher").getName
        }

    val amArgs =
      Seq(amClass) ++ userClass ++ userJar ++ primaryPyFile ++ primaryRFile ++
userArgs ++
      Seq("--properties-file",
        buildPath(Environment.PWD.$$(), LOCALIZED_CONF_DIR,
SPARK_CONF_FILE)) ++
      Seq("--dist-cache-conf",
        buildPath(Environment.PWD.$$(), LOCALIZED_CONF_DIR,
DIST_CACHE_CONF_FILE))

    // Command for the ApplicationMaster
    val commands = prefixEnv ++
      Seq(Environment.JAVA_HOME.$$() + "/bin/java", "-server") ++
      javaOpts ++ amArgs ++
      Seq(
        "1>", ApplicationConstants.LOG_DIR_EXPANSION_VAR + "/stdout",
        "2>", ApplicationConstants.LOG_DIR_EXPANSION_VAR + "/stderr")

    val printableCommands = commands.map(s => if (s == null) "null" else s).toList
    amContainer.setCommands(printableCommands.asJava)

    ... ...
    val securityManager = new SecurityManager(sparkConf)
    amContainer.setApplicationACLs(
      YarnSparkHadoopUtil.getApplicationAclsForYarn(securityManager).asJava)
    setupSecurityToken(amContainer)
    amContainer
}
```

1.3 ApplicationMaster 任务

1）在 IDEA 中全局查找（ctrl + n）org.apache.spark.deploy.yarn.ApplicationMaster**，点击对应的伴生对象**

ApplicationMaster.scala

```
def main(args: Array[String]): Unit = {

    // 1 解析传递过来的参数
    val amArgs = new ApplicationMasterArguments(args)
    val sparkConf = new SparkConf()
    ... ...

    val yarnConf = new
YarnConfiguration(SparkHadoopUtil.newConfiguration(sparkConf))
    // 2 创建 ApplicationMaster 对象
    master = new ApplicationMaster(amArgs, sparkConf, yarnConf)
    ... ...
    ugi.doAs(new PrivilegedExceptionAction[Unit]() {
        // 3 执行 ApplicationMaster
        override def run(): Unit = System.exit(master.run())
    })
```

```
}
```

1.3.1 解析传递过来的参数

ApplicationMasterArguments.scala

```
class ApplicationMasterArguments(val args: Array[String]) {
    ... ...
    parseArgs(args.toList)

    private def parseArgs(inputArgs: List[String]): Unit = {
        val userArgsBuffer = new ArrayBuffer[String]()
        var args = inputArgs

        while (!args.isEmpty) {
            args match {
                case ("--jar") :: value :: tail =>
                userJar = value
                args = tail

                case ("--class") :: value :: tail =>
                userClass = value
                args = tail
                ... ...

                case _ =>
                printUsageAndExit(1, args)
            }
        }
        ... ...
    }
    ... ...
}
```

1.3.2 创建 RMClient 并启动 Driver

ApplicationMaster.scala

```
private[spark] class ApplicationMaster(
    args: ApplicationMasterArguments,
    sparkConf: SparkConf,
    yarnConf: YarnConfiguration) extends Logging {
    ... ...
    // 1 创建 RMClient
    private val client = new YarnRMClient()
    ... ...
    final def run(): Int = {
        ... ...
        if (isClusterMode) {
            runDriver()
        } else {
            runExecutorLauncher()
        }
        ... ...
    }

    private def runDriver(): Unit = {
```

```
            addAmIpFilter(None,
System.getenv(ApplicationConstants.APPLICATION_WEB_PROXY_BASE_ENV))
            // 2 根据输入参数启动 Driver
            userClassThread = startUserApplication()

            val totalWaitTime = sparkConf.get(AM_MAX_WAIT_TIME)

            try {
                // 3 等待初始化完毕
                val sc = ThreadUtils.awaitResult(sparkContextPromise.future,
                    Duration(totalWaitTime, TimeUnit.MILLISECONDS))
               // sparkcontext 初始化完毕
                if (sc != null) {
                    val rpcEnv = sc.env.rpcEnv

                    val userConf = sc.getConf
                    val host = userConf.get(DRIVER_HOST_ADDRESS)
                    val port = userConf.get(DRIVER_PORT)
                    // 4 向 RM 注册自己（AM）
                    registerAM(host, port, userConf, sc.ui.map(_.webUrl), appAttemptId)

                    val driverRef = rpcEnv.setupEndpointRef(
                    RpcAddress(host, port),
                    YarnSchedulerBackend.ENDPOINT_NAME)
                    // 5 获取 RM 返回的可用资源列表
                    createAllocator(driverRef, userConf, rpcEnv, appAttemptId,
distCacheConf)
                } else {
                    ... ...
                }
                resumeDriver()
                userClassThread.join()
            } catch {
                ... ...
            } finally {
                resumeDriver()
            }
        }
```

ApplicationMaster.scala

```
private def startUserApplication(): Thread = {
    ... ...
    // args.userClass 来源于 ApplicationMasterArguments.scala
    val mainMethod = userClassLoader.loadClass(args.userClass)
    .getMethod("main", classOf[Array[String]])
    ... ...
    val userThread = new Thread {
        override def run(): Unit = {
            ... ...
            if (!Modifier.isStatic(mainMethod.getModifiers)) {
                logError(s"Could not find static main method in object
${args.userClass}")
                finish(FinalApplicationStatus.FAILED,
ApplicationMaster.EXIT_EXCEPTION_USER_CLASS)
```

```
                } else {
                    mainMethod.invoke(null, userArgs.toArray)
                    finish(FinalApplicationStatus.SUCCEEDED,
ApplicationMaster.EXIT_SUCCESS)
                    logDebug("Done running user class")
                }
                ... ...
            }
        }
        userThread.setContextClassLoader(userClassLoader)
        userThread.setName("Driver")
        userThread.start()
        userThread
}
```

1.3.3 向 RM 注册 AM

```
private def registerAM(
        host: String,
        port: Int,
        _sparkConf: SparkConf,
        uiAddress: Option[String],
        appAttempt: ApplicationAttemptId): Unit = {
        … …

        client.register(host, port, yarnConf, _sparkConf, uiAddress, historyAddress)
        registered = true
}
```

1.3.4 获取 RM 返回可以资源列表

ApplicationMaster.scala

```
private def createAllocator(
        driverRef: RpcEndpointRef,
        _sparkConf: SparkConf,
        rpcEnv: RpcEnv,
        appAttemptId: ApplicationAttemptId,
        distCacheConf: SparkConf): Unit = {

        ... ...
        // 申请资源 获得资源
        allocator = client.createAllocator(
        yarnConf,
        _sparkConf,
        appAttemptId,
        driverUrl,
        driverRef,
        securityMgr,
        localResources)

        ... ...
        // 处理资源结果，启动 Executor
        allocator.allocateResources()
        ... ...
}
```

YarnAllocator.scala

```
def allocateResources(): Unit = synchronized {
    val progressIndicator = 0.1f

    val allocateResponse = amClient.allocate(progressIndicator)
    // 获取可分配资源
    val allocatedContainers = allocateResponse.getAllocatedContainers()
    allocatorBlacklistTracker.setNumClusterNodes(allocateResponse.getNumClusterNodes)
    // 可分配的资源大于 0
    if (allocatedContainers.size > 0) {
        ......
        // 分配规则
        handleAllocatedContainers(allocatedContainers.asScala)
    }
    ... ...
}

def handleAllocatedContainers(allocatedContainers: Seq[Container]): Unit = {
    val containersToUse = new ArrayBuffer[Container](allocatedContainers.size)

    // 分配在同一台主机上资源
    val remainingAfterHostMatches = new ArrayBuffer[Container]
    for (allocatedContainer <- allocatedContainers) {
        ... ...
    }

    // 分配同一个机架上资源
    val remainingAfterRackMatches = new ArrayBuffer[Container]
    if (remainingAfterHostMatches.nonEmpty) {
        ... ...
    }

    // 分配既不是本地节点也不是机架本地的剩余部分
    val remainingAfterOffRackMatches = new ArrayBuffer[Container]
    for (allocatedContainer <- remainingAfterRackMatches) {
        ... ...
    }

    // 运行已分配容器
    runAllocatedContainers(containersToUse)
}
```

1.3.5 根据可用资源创建 NMClient

YarnAllocator.scala

```
private def runAllocatedContainers(containersToUse: ArrayBuffer[Container]): Unit = {

    for (container <- containersToUse) {
        ... ...
        if (runningExecutors.size() < targetNumExecutors) {
            numExecutorsStarting.incrementAndGet()
            if (launchContainers) {
                launcherPool.execute(() => {
                    try {
                        new ExecutorRunnable(
                            … …
```

```
                                ).run()
                                updateInternalState()
                            } catch {
                                ... ...
                            }
                        })
                    } else {
                        // For test only
                        updateInternalState()
                    }
                } else {
                    … …
                }
            }
}
```

ExecutorRunnable.scala

```
private[yarn] class ExecutorRunnable(... ...) extends Logging {
    var rpc: YarnRPC = YarnRPC.create(conf)
    var nmClient: NMClient = _

    def run(): Unit = {
        logDebug("Starting Executor Container")
        nmClient = NMClient.createNMClient()
        nmClient.init(conf)
        nmClient.start()
        startContainer()
    }
    ... ...
    def startContainer(): java.util.Map[String, ByteBuffer] = {
        ... ...
        // 准备命令，封装到 ctx 环境中
        val commands = prepareCommand()
        ctx.setCommands(commands.asJava)
        ... ...

        // 向指定的 NM 启动容器对象
        try {
            nmClient.startContainer(container.get, ctx)
        } catch {
            ... ...
        }
    }

    private def prepareCommand(): List[String] = {
        ... ...
        YarnSparkHadoopUtil.addOutOfMemoryErrorArgument(javaOpts)
        val commands = prefixEnv ++
        Seq(Environment.JAVA_HOME.$$() + "/bin/java", "-server") ++
        javaOpts ++
        Seq("org.apache.spark.executor.YarnCoarseGrainedExecutorBackend",
            "--driver-url", masterAddress,
            "--executor-id", executorId,
            "--hostname", hostname,
            "--cores", executorCores.toString,
            "--app-id", appId,
```

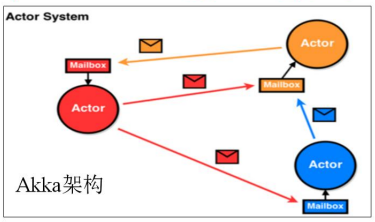
```
                "--resourceProfileId", resourceProfileId.toString) ++
            ... ...
        }
}
```

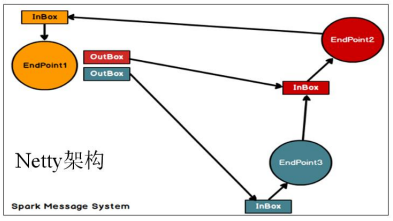
1.4 Spark 组件通信

1.4.1 Spark 中通信框架的发展

Spark中通信框架的发展

- Spark早期版本中采用Akka作为内部通信部件。
- Spark1.3中引入Netty通信框架，为了解决Shuffle的大数据传输问题使用
- Spark1.6中Akka和Netty可以配置使用。Netty完全实现了Akka在Spark中的功能。
- Spark2.x系列中，Spark抛弃Akka，使用Netty。





那么Netty为什么可以取代Akka？首先不容置疑的是Akka可以做到的，Netty也可以做到，但是Netty可以做到，Akka却无法做到，原因是啥？在软件栈中，Akka相比Netty要Higher一点，它专门针对RPC做了很多事情，而Netty相比更加基础一点，可以为不同的应用层通信协议（RPC，FTP，HTTP等）提供支持，在早期的Akka版本，底层的NIO通信就是用的Netty；其次一个优雅的工程师是不会允许一个系统中容纳两套通信框架，恶心！最后，虽然Netty没有Akka协程级的性能优势，但是Netty内部高效的Reactor线程模型，无锁化的串行设计，高效的序列化，零拷贝，内存池等特性也保证了Netty不会存在性能问题。

Endpoint有1个InBox和N个OutBox（N>=1，N取决于当前Endpoint与多少其他的Endpoint进行通信，一个与其通讯的其他Endpoint对应一个OutBox），Endpoint接收到的消息被写入InBox，发送出去的消息写入OutBox并被发送到其他Endpoint的InBox中。

1.4.2 三种通信方式

三种通信方式

1）三种通信模式

BIO：阻塞式IO

NIO：非阻塞式IO

AIO：异步非阻塞式IO

Spark底层采用Netty

Netty：支持NIO和Epoll模式

默认采用NIO

2）举例说明：

比如去饭店吃饭，老板说你前面有4个人，需要等一会：

（1）那你在桌子前一直等着，就是阻塞式IO——BIO。

（2）如果你和老板说，饭先做着，我先去打会篮球。在打篮球的过程中你时不时的回来看一下饭是否做好，就是非阻塞式IO——NIO。

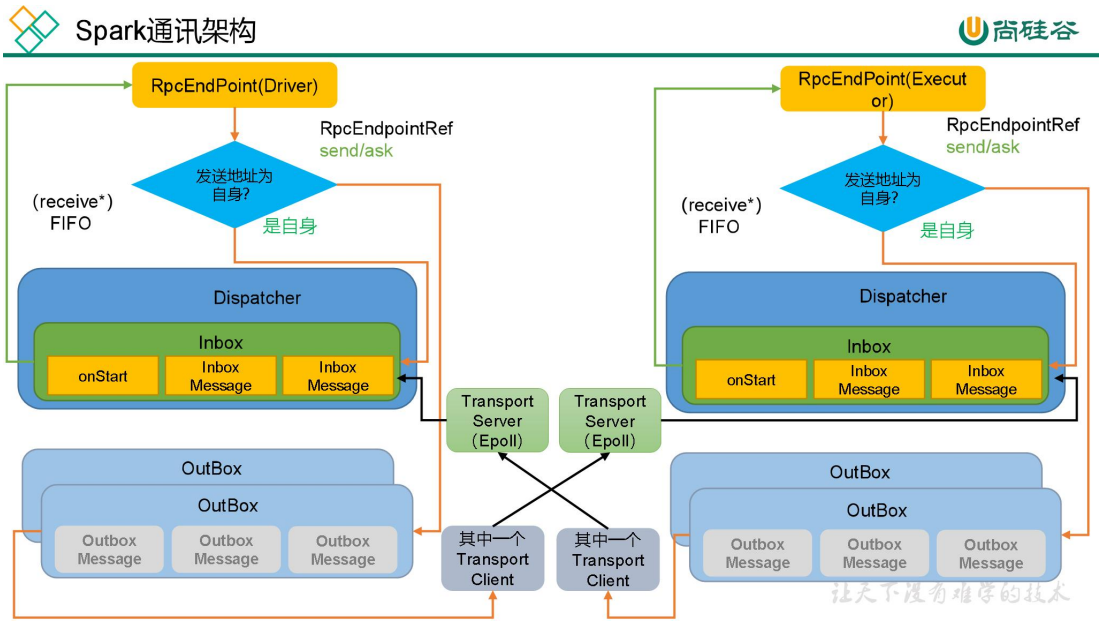
（3）先给老板说，我去打篮球，一个小时后给我送到指定位置，就是异步非阻塞式——AIO。

3）注意：

Linux对AIO支持的不够好，Windows支持AIO很好

Linux采用Epoll方式模仿AIO操作

1.4.3 Spark 底层通信原理



- **RpcEndpoint：RPC 通信终端**。Spark 针对每个节点（Client/Master/Worker）都称之为一个 RPC 终端，且都实现 RpcEndpoint 接口，内部根据不同端点的需求，设计不同的消息和不同的业务处理，如果需要发送（询问）则调用 Dispatcher。在 Spark 中，所有的终端都存在生命周期：

  Constructor =》**onStart** =》**receive*** =》onStop

- **RpcEnv：RPC 上下文环境**，每个 RPC 终端运行时依赖的上下文环境称为 RpcEnv；在当前 Spark 版本中使用的 NettyRpcEnv
- **Dispatcher：消息调度（分发）器**，针对于 RPC 终端需要发送远程消息或者从远程 RPC 接收到的消息，分发至对应的指令收件箱（发件箱）。如果指令接收方是自己则存入收件箱，如果指令接收方不是自己，则放入发件箱；
- **Inbox：指令消息收件箱。**一个本地 RpcEndpoint 对应一个收件箱，Dispatcher 在每次向 Inbox 存入消息时，都将对应 EndpointData 加入内部 ReceiverQueue 中，另外 Dispatcher 创建时会启动一个单独线程进行轮询 ReceiverQueue，进行收件箱消息消费；
- **RpcEndpointRef：RpcEndpointRef 是对远程 RpcEndpoint 的一个引用。**当我们需要向一个具体的 RpcEndpoint 发送消息时，一般我们需要获取到该 RpcEndpoint 的引用，然后通过该应用发送消息。
- **OutBox：指令消息发件箱。**对于当前 RpcEndpoint 来说，一个目标 RpcEndpoint 对应一个发件箱，如果向多个目标 RpcEndpoint 发送信息，则有多个 OutBox。当消息放入 Outbox

后，紧接着通过 TransportClient 将消息发送出去。消息放入发件箱以及发送过程是在同一个线程中进行；

- **RpcAddress：表示远程的 RpcEndpointRef 的地址**，Host + Port。
- **TransportClient：Netty 通信客户端**，一个 OutBox 对应一个 TransportClient，TransportClient 不断轮询 OutBox，根据 OutBox 消息的 receiver 信息，请求对应的远程 TransportServer；
- **TransportServer：Netty 通信服务端**，一个 RpcEndpoint 对应一个 TransportServer，接受远程消息后调用 Dispatcher 分发消息至对应收发件箱；

1.4.4 Executor 通信终端

1）在 IDEA 中全局查找（ctrl + n）org.apache.spark.executor.YarnCoarseGrainedExecutorBackend，**点击对应的伴生对象**

2）YarnCoarseGrainedExecutorBackend.scala 继承 CoarseGrainedExecutorBackend 继承 RpcEndpoint

```
// constructor -> onStart -> receive* -> onStop
private[spark] trait RpcEndpoint {

  val rpcEnv: RpcEnv

  final def self: RpcEndpointRef = {
    require(rpcEnv != null, "rpcEnv has not been initialized")
    rpcEnv.endpointRef(this)
  }

  def receive: PartialFunction[Any, Unit] = {
    case _ => throw new SparkException(self + " does not implement 'receive'")
  }

  def receiveAndReply(context: RpcCallContext): PartialFunction[Any, Unit] = {
    case _ => context.sendFailure(new SparkException(self + " won't reply anything"))
  }

  def onStart(): Unit = {
    // By default, do nothing.
  }

  def onStop(): Unit = {
    // By default, do nothing.
  }
}

private[spark] abstract class RpcEndpointRef(conf: SparkConf)
  extends Serializable with Logging {
  ... ...
```

```
    def send(message: Any): Unit
    def ask[T: ClassTag](message: Any, timeout: RpcTimeout): Future[T]
    ... ...
}
```

1.4.5 Driver 通信终端

ExecutorBackend 发送向 Driver 发送请求后，Driver 开始接收消息。全局查找（ctrl + n）SparkContext 类

SparkContext.scala

```
class SparkContext(config: SparkConf) extends Logging {
    ... ...
    private var _schedulerBackend: SchedulerBackend = _
    ... ...
}
```

点击 SchedulerBackend 进入 SchedulerBackend.scala，查找实现类（ctrl+h），找到 CoarseGrainedSchedulerBackend.scala，在该类内部创建 DriverEndpoint 对象。

```
private[spark]
class CoarseGrainedSchedulerBackend(scheduler: TaskSchedulerImpl, val rpcEnv: RpcEnv)
    extends ExecutorAllocationClient with SchedulerBackend with Logging {

    class DriverEndpoint extends IsolatedRpcEndpoint with Logging {
        override def receive: PartialFunction[Any, Unit] = {
            ... ...
            // 接收注册成功后的消息
            case LaunchedExecutor(executorId) =>
            executorDataMap.get(executorId).foreach { data =>
                data.freeCores = data.totalCores
            }
            makeOffers(executorId)
        }

        // 接收 ask 消息，并回复
        override def receiveAndReply(context: RpcCallContext): PartialFunction[Any,
Unit] = {

            case RegisterExecutor(executorId, executorRef, hostname, cores, logUrls,
                attributes, resources, resourceProfileId) =>
            ... ...
            context.reply(true)
            ... ...
        }
        ... ...
    }

    val driverEndpoint = rpcEnv.setupEndpoint(ENDPOINT_NAME,
createDriverEndpoint())

    protected def createDriverEndpoint(): DriverEndpoint = new DriverEndpoint()
}
```

DriverEndpoint 继承 IsolatedRpcEndpoint 继承 RpcEndpoint

```
// constructor -> onStart -> receive* -> onStop
private[spark] trait RpcEndpoint {

  val rpcEnv: RpcEnv

  final def self: RpcEndpointRef = {
    require(rpcEnv != null, "rpcEnv has not been initialized")
    rpcEnv.endpointRef(this)
  }

  def receive: PartialFunction[Any, Unit] = {
    case _ => throw new SparkException(self + " does not implement 'receive'")
  }

  def receiveAndReply(context: RpcCallContext): PartialFunction[Any, Unit] = {
    case _ => context.sendFailure(new SparkException(self + " won't reply anything"))
  }

  def onStart(): Unit = {
    // By default, do nothing.
  }

  def onStop(): Unit = {
    // By default, do nothing.
  }
}

private[spark] abstract class RpcEndpointRef(conf: SparkConf)
  extends Serializable with Logging {
  ... ...
  def send(message: Any): Unit
  def ask[T: ClassTag](message: Any, timeout: RpcTimeout): Future[T]
  ... ...
}
```

1.5 Executor 通信环境准备

1.5.1 创建 RPC 通信环境

1)在 IDEA 中全局查找(ctrl + n)org.apache.spark.executor.YarnCoarseGrainedExecutorBackend，

点击对应的伴生对象

2）运行 CoarseGrainedExecutorBackend

YarnCoarseGrainedExecutorBackend.scala

```
private[spark] object YarnCoarseGrainedExecutorBackend extends Logging {

    def main(args: Array[String]): Unit = {
        val createFn: (RpcEnv, CoarseGrainedExecutorBackend.Arguments, SparkEnv,
ResourceProfile) =>
        CoarseGrainedExecutorBackend = { case (rpcEnv, arguments, env, resourceProfile)
=>
        new YarnCoarseGrainedExecutorBackend(rpcEnv, arguments.driverUrl,
arguments.executorId,
```

```
                arguments.bindAddress, arguments.hostname, arguments.cores,
arguments.userClassPath, env,
                arguments.resourcesFileOpt, resourceProfile)
            }
            val backendArgs = CoarseGrainedExecutorBackend.parseArguments(args,
            this.getClass.getCanonicalName.stripSuffix("$"))
            CoarseGrainedExecutorBackend.run(backendArgs, createFn)
            System.exit(0)
        }
}
```

CoarseGrainedExecutorBackend.scala

```
def run(
    arguments: Arguments,
    backendCreateFn: (RpcEnv, Arguments, SparkEnv, ResourceProfile) =>
        CoarseGrainedExecutorBackend): Unit = {

    SparkHadoopUtil.get.runAsSparkUser { () =>

        // Bootstrap to fetch the driver's Spark properties.
        val executorConf = new SparkConf
        val fetcher = RpcEnv.create(
            "driverPropsFetcher",
            arguments.bindAddress,
            arguments.hostname,
            -1,
            executorConf,
            new SecurityManager(executorConf),
            numUsableCores = 0,
            clientMode = true)
        … …
        driverConf.set(EXECUTOR_ID, arguments.executorId)
        val env = SparkEnv.createExecutorEnv(driverConf, arguments.executorId,
arguments.bindAddress,
            arguments.hostname, arguments.cores, cfg.ioEncryptionKey, isLocal = false)

        env.rpcEnv.setupEndpoint("Executor",
            backendCreateFn(env.rpcEnv, arguments, env, cfg.resourceProfile))
        arguments.workerUrl.foreach { url =>
            env.rpcEnv.setupEndpoint("WorkerWatcher", new WorkerWatcher(env.rpcEnv,
url))
        }
        env.rpcEnv.awaitTermination()
    }
}
```

3）点击 create，进入 RpcEnv.Scala

```
def create(
    name: String,
    bindAddress: String,
    advertiseAddress: String,
    port: Int,
    conf: SparkConf,
    securityManager: SecurityManager,
    numUsableCores: Int,
    clientMode: Boolean): RpcEnv = {
```

```
    val config = RpcEnvConfig(conf, name, bindAddress, advertiseAddress, port,
securityManager,
      numUsableCores, clientMode)
    new NettyRpcEnvFactory().create(config)
}
```

NettyRpcEnv.scala

```
private[rpc] class NettyRpcEnvFactory extends RpcEnvFactory with Logging {

    def create(config: RpcEnvConfig): RpcEnv = {
        ... ...
        val nettyEnv =
            new NettyRpcEnv(sparkConf, javaSerializerInstance,
config.advertiseAddress,
                config.securityManager, config.numUsableCores)
        if (!config.clientMode) {
            val startNettyRpcEnv: Int => (NettyRpcEnv, Int) = { actualPort =>
                nettyEnv.startServer(config.bindAddress, actualPort)
                (nettyEnv, nettyEnv.address.port)
            }
            try {
                Utils.startServiceOnPort(config.port, startNettyRpcEnv, sparkConf,
config.name). 1
            } catch {
                case NonFatal(e) =>
                nettyEnv.shutdown()
                throw e
            }
        }
        nettyEnv
    }
}
```

1.5.2 创建多个发件箱

NettyRpcEnv.scala

```
private[netty] class NettyRpcEnv(
    val conf: SparkConf,
    javaSerializerInstance: JavaSerializerInstance,
    host: String,
    securityManager: SecurityManager,
    numUsableCores: Int) extends RpcEnv(conf) with Logging {
    ... ...
    private val outboxes = new ConcurrentHashMap[RpcAddress, Outbox]()
    ... ...
}
```

1.5.3 启动 TransportServer

NettyRpcEnv.scala

```
def startServer(bindAddress: String, port: Int): Unit = {
    ... ...
    server = transportContext.createServer(bindAddress, port, bootstraps)
    dispatcher.registerRpcEndpoint(
        RpcEndpointVerifier.NAME, new RpcEndpointVerifier(this, dispatcher))
}
```

TransportContext.scala

```
public TransportServer createServer(
        String host, int port, List<TransportServerBootstrap> bootstraps) {
        return new TransportServer(this, host, port, rpcHandler, bootstraps);
}
```

TransportServer.java

```
public TransportServer(
        TransportContext context,
        String hostToBind,
        int portToBind,
        RpcHandler appRpcHandler,
        List<TransportServerBootstrap> bootstraps) {
        ... ...
        init(hostToBind, portToBind);
        ... ...
}

private void init(String hostToBind, int portToBind) {
        // 默认是 NIO 模式
        IOMode ioMode = IOMode.valueOf(conf.ioMode());

        EventLoopGroup bossGroup = NettyUtils.createEventLoop(ioMode, 1,
                conf.getModuleName() + "-boss");
        EventLoopGroup workerGroup = NettyUtils.createEventLoop(ioMode,
conf.serverThreads(), conf.getModuleName() + "-server");

        bootstrap = new ServerBootstrap()
                .group(bossGroup, workerGroup)
                .channel(NettyUtils.getServerChannelClass(ioMode))
                .option(ChannelOption.ALLOCATOR, pooledAllocator)
                .option(ChannelOption.SO_REUSEADDR, !SystemUtils.IS_OS_WINDOWS)
                .childOption(ChannelOption.ALLOCATOR, pooledAllocator);
        ... ...
}
```

NettyUtils.java

```
public static Class<? extends ServerChannel> getServerChannelClass(IOMode mode) {
        switch (mode) {
                case NIO:
                        return NioServerSocketChannel.class;
                case EPOLL:
                        return EpollServerSocketChannel.class;
                default:
                        throw new IllegalArgumentException("Unknown io mode: " + mode);
        }
}
```

1.5.4 注册通信终端 RpcEndpoint

NettyRpcEnv.scala

```
def startServer(bindAddress: String, port: Int): Unit = {
        ... ...
        server = transportContext.createServer(bindAddress, port, bootstraps)
        dispatcher.registerRpcEndpoint(
```

```
            RpcEndpointVerifier.NAME, new RpcEndpointVerifier(this, dispatcher))
}
```

1.5.5 创建 TransportClient

Dispatcher.scala

```
def registerRpcEndpoint(name: String, endpoint: RpcEndpoint): NettyRpcEndpointRef = {
    ... ...
    val endpointRef = new NettyRpcEndpointRef(nettyEnv.conf, addr, nettyEnv)
    ... ...
}

private[netty] class NettyRpcEndpointRef(... ...) extends RpcEndpointRef(conf) {
    ... ...
    @transient @volatile var client: TransportClient = _
    // 创建 TransportClient
    private[netty] def createClient(address: RpcAddress): TransportClient = {
      clientFactory.createClient(address.host, address.port)
    }

    private val clientFactory = transportContext.createClientFactory(createClientBootstraps())
    ... ...
}
```

1.5.6 收发邮件箱

1）接收邮件箱 1 个

Dispatcher.scala

```
def registerRpcEndpoint(name: String, endpoint: RpcEndpoint): NettyRpcEndpointRef = {
        ... ...
        var messageLoop: MessageLoop = null
        try {
            messageLoop = endpoint match {
            case e: IsolatedRpcEndpoint =>
                new DedicatedMessageLoop(name, e, this)
            case   =>
                sharedLoop.register(name, endpoint)
                sharedLoop
            }
            endpoints.put(name, messageLoop)
        } catch {
            ... ...
        }
    }
    endpointRef
}
```

DedicatedMessageLoop.scala

```
private class DedicatedMessageLoop(
    name: String,
    endpoint: IsolatedRpcEndpoint,
    dispatcher: Dispatcher)
  extends MessageLoop(dispatcher) {
```

```
    private val inbox = new Inbox(name, endpoint)
    … …
}
```

Inbox.scala

```
private[netty] class Inbox(val endpointName: String, val endpoint: RpcEndpoint)
  extends Logging {
    ... ...
    inbox.synchronized {
        messages.add(OnStart)
    }
    ... ...
}
```

1.6 Executor 注册

CoarseGrainedExecutorBackend.scala

```
// RPC 生命周期：  constructor -> onStart -> receive* -> onStop
private[spark] class CoarseGrainedExecutorBackend(... ...)
  extends IsolatedRpcEndpoint with ExecutorBackend with Logging {
    ... ...
    override def onStart(): Unit = {
        … …
        rpcEnv.asyncSetupEndpointRefByURI(driverUrl).flatMap { ref =>
        // This is a very fast action so we can use "ThreadUtils.sameThread"
        driver = Some(ref)
// 1 向 Driver 注册自己
        ref.ask[Boolean](RegisterExecutor(executorId, self, hostname, cores,
extractLogUrls, extractAttributes, _resources, resourceProfile.id))
        }(ThreadUtils.sameThread).onComplete {
// 2 接收 Driver 返回成功的消息，并给自己发送注册成功消息
        case Success(  ) =>
            self.send(RegisteredExecutor)
        case Failure(e) =>
            exitExecutor(1, s"Cannot register with driver: $driverUrl", e, notifyDriver =
false)
        }(ThreadUtils.sameThread)
    }
    ... ...

    override def receive: PartialFunction[Any, Unit] = {
// 3 收到注册成功的消息后，创建 Executor，并启动 Executor
        case RegisteredExecutor =>
        try {
            // 创建 Executor
            executor = new Executor(executorId, hostname, env, userClassPath, isLocal =
false, resources = _resources)
            driver.get.send(LaunchedExecutor(executorId))
        } catch {
            case NonFatal(e) =>
            exitExecutor(1, "Unable to create executor due to " + e.getMessage, e)
        }
        ... ...
    }
}
```

1.7 Driver 接收消息并应答

ExecutorBackend 发送向 Driver 发送请求后，Driver 开始接收消息。全局查找（ctrl + n）SparkContext 类

SparkContext.scala

```
class SparkContext(config: SparkConf) extends Logging {
    ... ...
    private var _schedulerBackend: SchedulerBackend = _
    ... ...
}
```

点击 SchedulerBackend 进入 SchedulerBackend.scala，查找实现类（ctrl+h），找到 CoarseGrainedSchedulerBackend.scala

```
private[spark]
class CoarseGrainedSchedulerBackend(scheduler: TaskSchedulerImpl, val rpcEnv: RpcEnv)
    extends ExecutorAllocationClient with SchedulerBackend with Logging {

    class DriverEndpoint extends IsolatedRpcEndpoint with Logging {
        override def receive: PartialFunction[Any, Unit] = {
            ... ...
            // 接收注册成功后的消息
            case LaunchedExecutor(executorId) =>
            executorDataMap.get(executorId).foreach { data =>
                data.freeCores = data.totalCores
            }
            makeOffers(executorId)
        }

        // 接收 ask 消息，并回复
        override def receiveAndReply(context: RpcCallContext): PartialFunction[Any, Unit] = {

            case RegisterExecutor(executorId, executorRef, hostname, cores, logUrls,
                attributes, resources, resourceProfileId) =>
            ... ...
            context.reply(true)
            ... ...
        }
        ... ...
    }

    val driverEndpoint = rpcEnv.setupEndpoint(ENDPOINT_NAME, createDriverEndpoint())

    protected def createDriverEndpoint(): DriverEndpoint = new DriverEndpoint()
}
```

1.8 Executor 执行代码

1.8.1 SparkContext 初始化完毕，通知执行后续代码

1）进入到 ApplicationMaster

ApplicationMaster.scala

```
private[spark] class ApplicationMaster(
    args: ApplicationMasterArguments,
    sparkConf: SparkConf,
    yarnConf: YarnConfiguration) extends Logging {

    private def runDriver(): Unit = {
        addAmIpFilter(None,
System.getenv(ApplicationConstants.APPLICATION_WEB_PROXY_BASE_ENV))
        userClassThread = startUserApplication()

        val totalWaitTime = sparkConf.get(AM_MAX_WAIT_TIME)
        try {
            val sc = ThreadUtils.awaitResult(sparkContextPromise.future,
                Duration(totalWaitTime, TimeUnit.MILLISECONDS))
            if (sc != null) {
                val rpcEnv = sc.env.rpcEnv

                val userConf = sc.getConf
                val host = userConf.get(DRIVER_HOST_ADDRESS)
                val port = userConf.get(DRIVER_PORT)
                registerAM(host, port, userConf, sc.ui.map(_.webUrl), appAttemptId)

                val driverRef = rpcEnv.setupEndpointRef(
                RpcAddress(host, port),
                YarnSchedulerBackend.ENDPOINT_NAME)
                createAllocator(driverRef, userConf, rpcEnv, appAttemptId,
distCacheConf)
            } else {
                … …
            }
            // 执行程序
            resumeDriver()
            userClassThread.join()
        } catch {
            ... ...
        } finally {
            resumeDriver()
        }
    }
    ... ...
    private def resumeDriver(): Unit = {
        sparkContextPromise.synchronized {
            sparkContextPromise.notify()
        }
    }
}
```

1.8.2 接收代码继续执行消息

在 SparkContext.scala 文件中查找_taskScheduler.postStartHook()，点击 postStartHook，查找实现类（ctrl + h）

```
private[spark] class YarnClusterScheduler(sc: SparkContext) extends YarnScheduler(sc) {

    logInfo("Created YarnClusterScheduler")

    override def postStartHook(): Unit = {
        ApplicationMaster.sparkContextInitialized(sc)
        super.postStartHook()
        logInfo("YarnClusterScheduler.postStartHook done")
    }
}
```

点击 super.postStartHook()

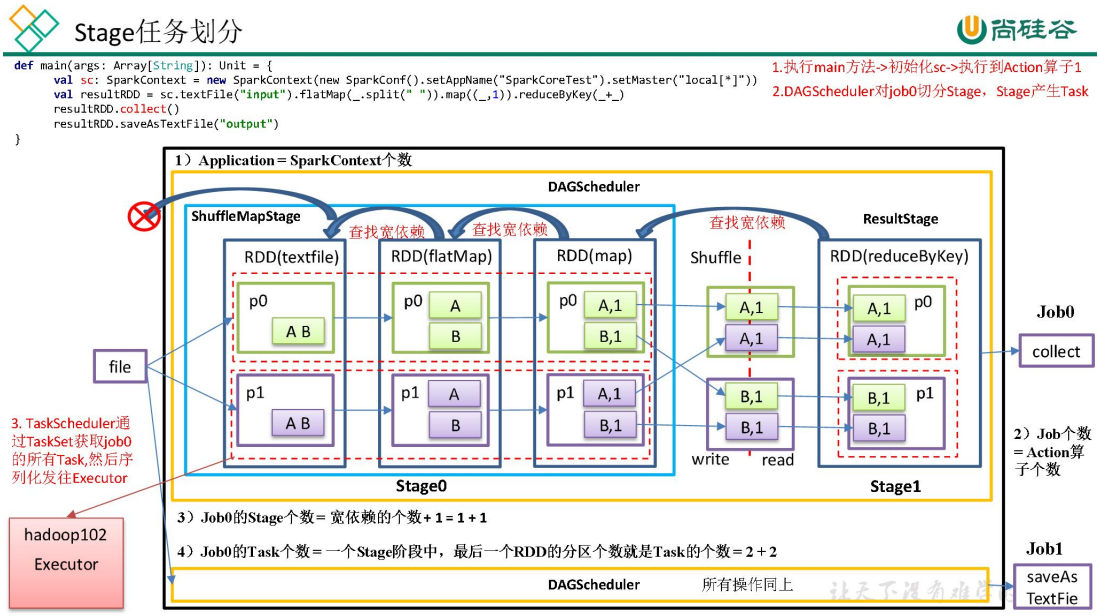
TaskSchedulerImpl.scala

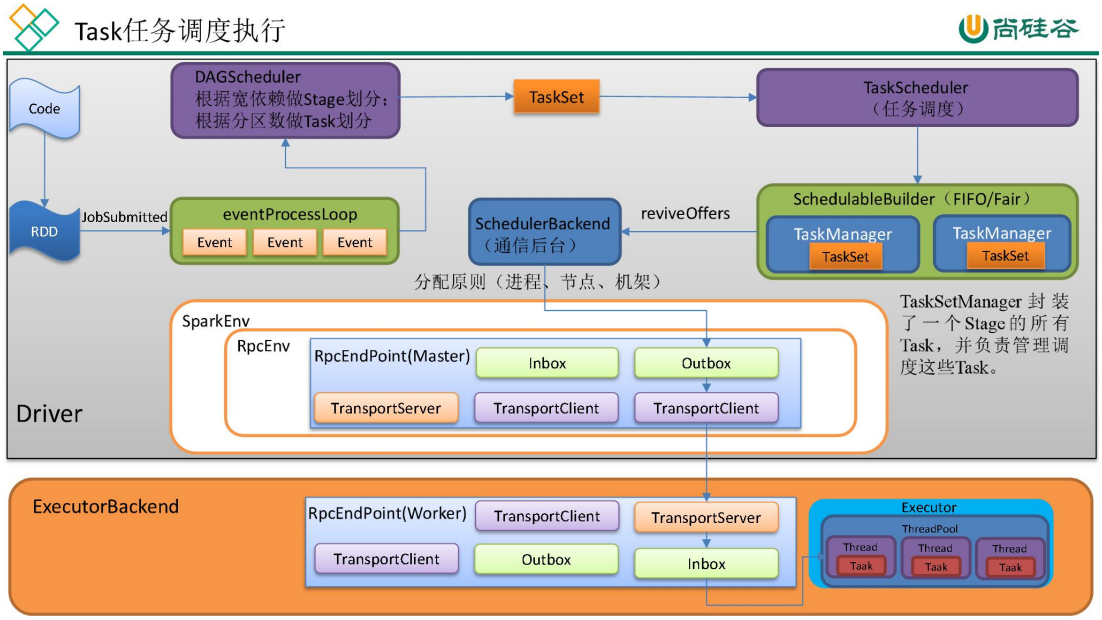
```
override def postStartHook(): Unit = {
    waitBackendReady()
}

private def waitBackendReady(): Unit = {
    if (backend.isReady) {
        return
    }
    while (!backend.isReady) {
        if (sc.stopped.get) {
            throw new IllegalStateException("Spark context stopped while waiting for backend")
        }
        synchronized {
            this.wait(100)
        }
    }
}
```

第 2 章 任务的执行

2.1 概述

2.1.1 任务切分和任务调度原理





Task FIFO调度算法

```
val priority1 = s1.priority
val priority2 = s2.priority
var res = math.signum(priority1 - priority2)
if (res == 0) {
  val stageId1 = s1.stageId
  val stageId2 = s2.stageId
  res = math.signum(stageId1 - stageId2)
}
res < 0
```

- **priority**(优先级)：生成 TaskSet实例时，传入的JobID
- **stageId**：生成 TaskSet实例时，传入的stage的ID

```
taskScheduler.submitTasks(new TaskSet(
  tasks.toArray, stage.id,
stage.latestInfo.attemptNumber, jobId,
properties))
```

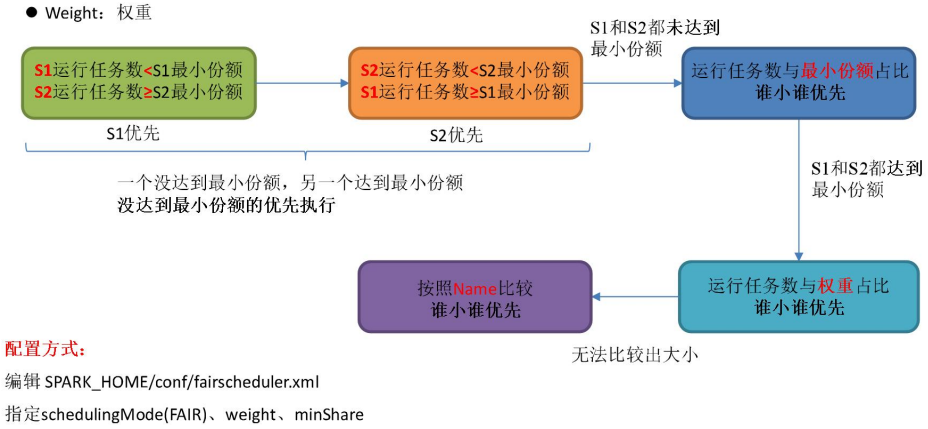
遇到一个行动算子生成一个Job，jobId是递增的

同一个job内部的stageId是递增的

- 先生成的Job（对应SPARK中代码就是先执行的ACTION)先进行调度
- 如果是同一个Job，最上层的STAGE先进行调度

Task公平调度算法

- runningTasks：运行中的任务数
- minShare：最小份额
- Weight：权重



配置方式：

编辑 SPARK_HOME/conf/fairscheduler.xml

指定schedulingMode(FAIR)、weight、minShare

2.1.2 本地化调度

任务分配原则：根据每个 Task 的优先位置，确定 Task 的 Locality（本地化）级别，本地化一共有五种，优先级由高到低顺序：

移动数据不如移动计算。

名称	解析
PROCESS_LOCAL	进程本地化，task 和数据在同一个 Executor 中，性能最好。
NODE_LOCAL	节点本地化，task 和数据在同一个节点中，但是 task 和数据不在同一个 Executor 中，数据需要在进程间进行传输。
RACK_LOCAL	机架本地化，task 和数据在同一个机架的两个节点上，数据需要通过网络在节点之间进行传输。

NO_PREF	对于 task 来说，从哪里获取都一样，没有好坏之分。
ANY	task 和数据可以在集群的任何地方，而且不在一个机架中，性能最差。

2.1.3 失败重试与黑名单机制

除了选择合适的 Task 调度运行外，还需要监控 Task 的执行状态，前面也提到，与外部打交道的是 SchedulerBackend，Task 被提交到 Executor 启动执行后，Executor 会将执行状态上报给 SchedulerBackend，SchedulerBackend 则告诉 TaskScheduler，TaskScheduler 找到该 Task 对应的 TaskSetManager，并通知到该 TaskSetManager，这样 TaskSetManager 就知道 Task 的失败与成功状态，对于失败的 Task，会记录它失败的次数，如果失败次数还没有超过最大重试次数，那么就把它放回待调度的 Task 池子中，否则整个 Application 失败。

在记录 Task 失败次数过程中，会记录它上一次失败所在的 Executor Id 和 Host，这样下次再调度这个 Task 时，会使用黑名单机制，避免它被调度到上一次失败的节点上，起到一定的容错作用。黑名单记录 Task 上一次失败所在的 Executor Id 和 Host，以及其对应的“拉黑”时间，“拉黑”时间是指这段时间内不要再往这个节点上调度这个 Task 了。

2.2 阶段的划分

0）在 WordCount 程序中查看源码

```
package com.atguigu.spark

import org.apache.spark.rdd.RDD
import org.apache.spark.{SparkConf, SparkContext}

object WordCount {

    def main(args: Array[String]): Unit = {

        // 1  创建 sc
        val conf: SparkConf = new SparkConf().setAppName("WC").setMaster("local[*]")

        val sc: SparkContext = new SparkContext(conf)

        // 2  读取数据  hello atguigu spark spark
        val lineRDD: RDD[String] = sc.textFile("input")

        // 3  一行变多行
        val wordRDD: RDD[String] = lineRDD.flatMap((x: String) => x.split(" "))

        // 4  变换结构    一行变一行
        val wordToOneRDD: RDD[(String, Int)] = wordRDD.map((x: String) => (x, 1))

        // 5    聚合 key 相同的单纯
        val wordToSumRDD: RDD[(String, Int)] = wordToOneRDD.reduceByKey((v1, v2)
=> v1 + v2)
```

```
        // 6 收集打印
        wordToSumRDD.collect().foreach(println)

        //7 关闭资源
        sc.stop()
    }
}
```

1）在 WordCount 代码中点击 collect

RDD.scala

```
def collect(): Array[T] = withScope {
    val results = sc.runJob(this, (iter: Iterator[T]) => iter.toArray)
    Array.concat(results: _*)
}
```

SparkContext.scala

```
def runJob[T, U: ClassTag](rdd: RDD[T], func: Iterator[T] => U): Array[U] = {
    runJob(rdd, func, 0 until rdd.partitions.length)
}

def runJob[T, U: ClassTag](
    rdd: RDD[T],
    func: Iterator[T] => U,
    partitions: Seq[Int]): Array[U] = {
    val cleanedFunc = clean(func)
    runJob(rdd, (ctx: TaskContext, it: Iterator[T]) => cleanedFunc(it), partitions)
}

def runJob[T, U: ClassTag](
    rdd: RDD[T],
    func: (TaskContext, Iterator[T]) => U,
    partitions: Seq[Int]): Array[U] = {
    val results = new Array[U](partitions.size)
    runJob[T, U](rdd, func, partitions, (index, res) => results(index) = res)
    results
}

def runJob[T, U: ClassTag](
    rdd: RDD[T],
    func: (TaskContext, Iterator[T]) => U,
    partitions: Seq[Int],
    resultHandler: (Int, U) => Unit): Unit = {
    ... ...
    dagScheduler.runJob(rdd, cleanedFunc, partitions, callSite, resultHandler,
localProperties.get)
    ... ...
}
```

DAGScheduler.scala

```
def runJob[T, U](
    rdd: RDD[T],
    func: (TaskContext, Iterator[T]) => U,
    partitions: Seq[Int],
    callSite: CallSite,
```

```
        resultHandler: (Int, U) => Unit,
        properties: Properties): Unit = {
        ... ...
        val waiter = submitJob(rdd, func, partitions, callSite, resultHandler, properties)
        ... ...
}

def submitJob[T, U](
        rdd: RDD[T],
        func: (TaskContext, Iterator[T]) => U,
        partitions: Seq[Int],
        callSite: CallSite,
        resultHandler: (Int, U) => Unit,
        properties: Properties): JobWaiter[U] = {
        ... ...
        val waiter = new JobWaiter[U](this, jobId, partitions.size, resultHandler)
        eventProcessLoop.post(JobSubmitted(
                jobId, rdd, func2, partitions.toArray, callSite, waiter,
                Utils.cloneProperties(properties)))
        waiter
}
```

EventLoop.scala

```
def post(event: E): Unit = {
        if (!stopped.get) {
                if (eventThread.isAlive) {
                        eventQueue.put(event)
                } else {
                        ... ...
                }
        }
}

private[spark] val eventThread = new Thread(name) {

        override def run(): Unit = {
                while (!stopped.get) {
                        val event = eventQueue.take()
                        try {
                                onReceive(event)
                        } catch {
                                ... ...
                        }
                }
        }
}
```

查找 onReceive 实现类（ctrl + h）

DAGScheduler.scala

```
private[scheduler] class DAGSchedulerEventProcessLoop(dagScheduler: DAGScheduler)
        extends EventLoop[DAGSchedulerEvent]("dag-scheduler-event-loop") with Logging {

        ... ...
        override def onReceive(event: DAGSchedulerEvent): Unit = {
                val timerContext = timer.time()
```

```
            try {
                doOnReceive(event)
            } finally {
                timerContext.stop()
            }
        }

        private def doOnReceive(event: DAGSchedulerEvent): Unit = event match {
            case JobSubmitted(jobId, rdd, func, partitions, callSite, listener, properties) =>
            dagScheduler.handleJobSubmitted(jobId, rdd, func, partitions, callSite, listener,
properties)
            ... ...
        }
        ... ...
        private[scheduler] def handleJobSubmitted(jobId: Int,
            finalRDD: RDD[_],
            func: (TaskContext, Iterator[_]) => _,
            partitions: Array[Int],
            callSite: CallSite,
            listener: JobListener,
            properties: Properties): Unit = {

            var finalStage: ResultStage = null

            finalStage = createResultStage(finalRDD, func, partitions, jobId, callSite)
            ... ...
        }

        private def createResultStage(
            rdd: RDD[_],
            func: (TaskContext, Iterator[_]) => _,
            partitions: Array[Int],
            jobId: Int,
            callSite: CallSite): ResultStage = {
            … …
            val parents = getOrCreateParentStages(rdd, jobId)
            val id = nextStageId.getAndIncrement()

            val stage = new ResultStage(id, rdd, func, partitions, parents, jobId, callSite)
            stageIdToStage(id) = stage
            updateJobIdStageIdMaps(jobId, stage)
            stage
        }

        private def getOrCreateParentStages(rdd: RDD[_], firstJobId: Int): List[Stage] = {
            getShuffleDependencies(rdd).map { shuffleDep =>
                getOrCreateShuffleMapStage(shuffleDep, firstJobId)
            }.toList
        }

        private[scheduler] def getShuffleDependencies(
            rdd: RDD[  ]): HashSet[ShuffleDependency[  ,  ,  ]] = {

                val parents = new HashSet[ShuffleDependency[_, _, _]]
                val visited = new HashSet[RDD[_]]
                val waitingForVisit = new ListBuffer[RDD[_]]
```

```
                waitingForVisit += rdd
                while (waitingForVisit.nonEmpty) {
                val toVisit = waitingForVisit.remove(0)

                if (!visited(toVisit)) {
                        visited += toVisit
                        toVisit.dependencies.foreach {
                        case shuffleDep: ShuffleDependency[_, _, _] =>
                                parents += shuffleDep
                        case dependency =>
                                waitingForVisit.prepend(dependency.rdd)
                        }
                }
        }
        parents
    }

    private def getOrCreateShuffleMapStage(
        shuffleDep: ShuffleDependency[_, _, _],
        firstJobId: Int): ShuffleMapStage = {

        shuffleIdToMapStage.get(shuffleDep.shuffleId) match {
            case Some(stage) =>
                stage

            case None =>
                getMissingAncestorShuffleDependencies(shuffleDep.rdd).foreach { dep
=>

                    if (!shuffleIdToMapStage.contains(dep.shuffleId)) {
                        createShuffleMapStage(dep, firstJobId)
                    }
                }
                // Finally, create a stage for the given shuffle dependency.
                createShuffleMapStage(shuffleDep, firstJobId)
        }
    }

    def createShuffleMapStage[K, V, C](
        shuffleDep: ShuffleDependency[K, V, C], jobId: Int): ShuffleMapStage = {
        ... ...
        val rdd = shuffleDep.rdd
        val numTasks = rdd.partitions.length
        val parents = getOrCreateParentStages(rdd, jobId)
        val id = nextStageId.getAndIncrement()

        val stage = new ShuffleMapStage(
            id, rdd, numTasks, parents, jobId, rdd.creationSite, shuffleDep,
mapOutputTracker)
        ... ...
    }
    ... ...
}
```

2.3 任务的切分

DAGScheduler.scala

```
private[scheduler] def handleJobSubmitted(jobId: Int,
      finalRDD: RDD[_],
      func: (TaskContext, Iterator[_]) => _,
      partitions: Array[Int],
      callSite: CallSite,
      listener: JobListener,
      properties: Properties): Unit = {

      var finalStage: ResultStage = null
      try {
            finalStage = createResultStage(finalRDD, func, partitions, jobId, callSite)
      } catch {
            ... ...
      }

      val job = new ActiveJob(jobId, finalStage, callSite, listener, properties)
      ... ...
      submitStage(finalStage)
}

private def submitStage(stage: Stage): Unit = {
      val jobId = activeJobForStage(stage)

      if (jobId.isDefined) {
            if (!waitingStages(stage) && !runningStages(stage) && !failedStages(stage)) {
                  val missing = getMissingParentStages(stage).sortBy(_.id)

                  if (missing.isEmpty) {
                        submitMissingTasks(stage, jobId.get)
                  } else {
                        for (parent <- missing) {
                              submitStage(parent)
                        }
                        waitingStages += stage
                  }
            }
      } else {
            abortStage(stage, "No active job for stage " + stage.id, None)
      }
}

private def submitMissingTasks(stage: Stage, jobId: Int): Unit = {

      val partitionsToCompute: Seq[Int] = stage.findMissingPartitions()
      ... ...
      val tasks: Seq[Task[_]] = try {
            val                                  serializedTaskMetrics                                =
closureSerializer.serialize(stage.latestInfo.taskMetrics).array()

            stage match {
            case stage: ShuffleMapStage =>
```

```
                stage.pendingPartitions.clear()
                partitionsToCompute.map { id =>
                val locs = taskIdToLocations(id)
                val part = partitions(id)
                stage.pendingPartitions += id
                new ShuffleMapTask(stage.id, stage.latestInfo.attemptNumber,
                    taskBinary, part, locs, properties, serializedTaskMetrics, Option(jobId),
                    Option(sc.applicationId), sc.applicationAttemptId, stage.rdd.isBarrier())
                }
            case stage: ResultStage =>
                partitionsToCompute.map { id =>
                val p: Int = stage.partitions(id)
                val part = partitions(p)
                val locs = taskIdToLocations(id)
                new ResultTask(stage.id, stage.latestInfo.attemptNumber,
                    taskBinary, part, locs, id, properties, serializedTaskMetrics,
                    Option(jobId), Option(sc.applicationId), sc.applicationAttemptId,
                    stage.rdd.isBarrier())
                }
            }
        } catch {
            ... ...
        }
}
```

Stage.scala

```
private[scheduler] abstract class Stage(... ...)
    extends Logging {
    ... ...
    def findMissingPartitions(): Seq[Int]
    ... ...
}
```

全局查找（ctrl + h）findMissingPartitions 实现类。

ShuffleMapStage.scala

```
private[spark] class ShuffleMapStage(... ...)
    extends Stage(id, rdd, numTasks, parents, firstJobId, callSite) {

    private[this] var _mapStageJobs: List[ActiveJob] = Nil
    ... ...
    override def findMissingPartitions(): Seq[Int] = {
        mapOutputTrackerMaster
        .findMissingPartitions(shuffleDep.shuffleId)
        .getOrElse(0 until numPartitions)
    }
}
```

ResultStage.scala

```
private[spark] class ResultStage(... ...)
    extends Stage(id, rdd, partitions.length, parents, firstJobId, callSite) {
    ... ...
    override def findMissingPartitions(): Seq[Int] = {
        val job = activeJob.get(0 until job.numPartitions).filter(id => !job.finished(id))
    }
```

```
    ... ...
}
```

2.4 任务的调度

1）提交任务

DAGScheduler.scala

```
private def submitMissingTasks(stage: Stage, jobId: Int): Unit = {
    ... ...
    if (tasks.nonEmpty) {
        taskScheduler.submitTasks(new TaskSet( tasks.toArray, stage.id,
stage.latestInfo.attemptNumber, jobId, properties))
    } else {
        markStageAsFinished(stage, None)

        stage match {
        case stage: ShuffleMapStage =>

            markMapStageJobsAsFinished(stage)
        case stage : ResultStage =>
            logDebug(s"Stage ${stage} is actually done; (partitions:
${stage.numPartitions})")
        }
        submitWaitingChildStages(stage)
    }
}
```

TaskScheduler.scala

```
def submitTasks(taskSet: TaskSet): Unit
```

全局查找 submitTasks 的实现类 TaskSchedulerImpl

TaskSchedulerImpl.scala

```
override def submitTasks(taskSet: TaskSet): Unit = {
    val tasks = taskSet.tasks
    this.synchronized {
        val manager = createTaskSetManager(taskSet, maxTaskFailures)
        val stage = taskSet.stageId
        val stageTaskSets =
        taskSetsByStageIdAndAttempt.getOrElseUpdate(stage, new HashMap[Int,
TaskSetManager])
        ... ...
        stageTaskSets(taskSet.stageAttemptId) = manager
        // 向队列里面设置任务
        schedulableBuilder.addTaskSetManager(manager, manager.taskSet.properties)
        ... ...
    }
    // 取任务
    backend.reviveOffers()
}
```

2）FIFO 和公平调度器

点击 schedulableBuilder，查找 schedulableBuilder 初始化赋值的地方

```
private var schedulableBuilder: SchedulableBuilder = null
```

```
def initialize(backend: SchedulerBackend): Unit = {
    this.backend = backend
    schedulableBuilder = {
        schedulingMode match {
        case SchedulingMode.FIFO =>
            new FIFOSchedulableBuilder(rootPool)
        case SchedulingMode.FAIR =>
            new FairSchedulableBuilder(rootPool, conf)
        case _ =>
            throw new IllegalArgumentException(s"Unsupported
$SCHEDULER_MODE_PROPERTY: " +
            s"$schedulingMode")
        }
    }
    schedulableBuilder.buildPools()
}
```

点击 schedulingMode，default scheduler is FIFO

```
private val schedulingModeConf = conf.get(SCHEDULER_MODE)
val schedulingMode: SchedulingMode =
    ... ...
      SchedulingMode.withName(schedulingModeConf.toUpperCase(Locale.ROOT))
    ... ...
}
private[spark] val SCHEDULER_MODE =
ConfigBuilder("spark.scheduler.mode")
    .version("0.8.0")
    .stringConf
    .createWithDefault(SchedulingMode.FIFO.toString)
```

3）读取任务

SchedulerBackend.scala

```
private[spark] trait SchedulerBackend {
    ... ...
    def reviveOffers(): Unit
    ... ...
}
```

全局查找 reviveOffers 实现类 CoarseGrainedSchedulerBackend

CoarseGrainedSchedulerBackend.scala

```
override def reviveOffers(): Unit = {
    // 自己给自己发消息
    driverEndpoint.send(ReviveOffers)
}
// 自己接收到消息
override def receive: PartialFunction[Any, Unit] = {
    ... ...
    case ReviveOffers =>
        makeOffers()
    ... ...
}

private def makeOffers(): Unit = {
```

```
        val taskDescs = withLock {
            ... ...
            // 取任务
            scheduler.resourceOffers(workOffers)
        }
        if (taskDescs.nonEmpty) {
            launchTasks(taskDescs)
        }
    }
```

TaskSchedulerImpl.scala

```
def resourceOffers(offers: IndexedSeq[WorkerOffer]): Seq[Seq[TaskDescription]] =
synchronized {
    ... ...
    val sortedTaskSets = rootPool.getSortedTaskSetQueue.filterNot(_.isZombie)

    for (taskSet <- sortedTaskSets) {
        val availableSlots = availableCpus.map(c => c / CPUS_PER_TASK).sum

        if (taskSet.isBarrier && availableSlots < taskSet.numTasks) {

        } else {
            var launchedAnyTask = false
            val addressesWithDescs = ArrayBuffer[(String, TaskDescription)]()

            for (currentMaxLocality <- taskSet.myLocalityLevels) {
                var launchedTaskAtCurrentMaxLocality = false
                do {
                    launchedTaskAtCurrentMaxLocality =
resourceOfferSingleTaskSet(taskSet,
                        currentMaxLocality, shuffledOffers, availableCpus,
                        availableResources, tasks, addressesWithDescs)
                    launchedAnyTask |= launchedTaskAtCurrentMaxLocality
                } while (launchedTaskAtCurrentMaxLocality)
            }
            ... ...
        }
    }
    ... ...
    return tasks
}
```

Pool.scala

```
override def getSortedTaskSetQueue: ArrayBuffer[TaskSetManager] = {
    val sortedTaskSetQueue = new ArrayBuffer[TaskSetManager]
    val sortedSchedulableQueue =

schedulableQueue.asScala.toSeq.sortWith(taskSetSchedulingAlgorithm.comparator)

    for (schedulable <- sortedSchedulableQueue) {
        sortedTaskSetQueue ++= schedulable.getSortedTaskSetQueue
    }
    sortedTaskSetQueue
}
```

```
private val taskSetSchedulingAlgorithm: SchedulingAlgorithm = {
    schedulingMode match {
        case SchedulingMode.FAIR =>
            new FairSchedulingAlgorithm()
        case SchedulingMode.FIFO =>
            new FIFOSchedulingAlgorithm()
        case _ =>
            … …
    }
}
```

4）FIFO 和公平调度器规则

SchedulingAlgorithm.scala

```
private[spark] class FIFOSchedulingAlgorithm extends SchedulingAlgorithm {
    override def comparator(s1: Schedulable, s2: Schedulable): Boolean = {
        val priority1 = s1.priority
        val priority2 = s2.priority
        var res = math.signum(priority1 - priority2)
        if (res == 0) {
            val stageId1 = s1.stageId
            val stageId2 = s2.stageId
            res = math.signum(stageId1 - stageId2)
        }
        res < 0
    }
}

private[spark] class FairSchedulingAlgorithm extends SchedulingAlgorithm {
    override def comparator(s1: Schedulable, s2: Schedulable): Boolean = {
        val minShare1 = s1.minShare
        val minShare2 = s2.minShare
        val runningTasks1 = s1.runningTasks
        val runningTasks2 = s2.runningTasks
        val s1Needy = runningTasks1 < minShare1
        val s2Needy = runningTasks2 < minShare2
        val minShareRatio1 = runningTasks1.toDouble / math.max(minShare1, 1.0)
        val minShareRatio2 = runningTasks2.toDouble / math.max(minShare2, 1.0)
        val taskToWeightRatio1 = runningTasks1.toDouble / s1.weight.toDouble
        val taskToWeightRatio2 = runningTasks2.toDouble / s2.weight.toDouble
        … …
    }
}
```

5）发送给 Executor 端执行任务

CoarseGrainedSchedulerBackend.scala

```
private def makeOffers(): Unit = {

    val taskDescs = withLock {
        ... ...
        // 取任务
        scheduler.resourceOffers(workOffers)
    }
    if (taskDescs.nonEmpty) {
```

```
            launchTasks(taskDescs)
        }
    }

    private def launchTasks(tasks: Seq[Seq[TaskDescription]]): Unit = {

        for (task <- tasks.flatten) {
            val serializedTask = TaskDescription.encode(task)

            if (serializedTask.limit() >= maxRpcMessageSize) {
                ... ...
            }
            else {
                … …
                // 序列化任务发往 Executor 远程终端
                executorData.executorEndpoint.send(LaunchTask(new
SerializableBuffer(serializedTask)))
            }
        }
    }
```

2.5 任务的执行

在 CoarseGrainedExecutorBackend.scala 中接收数据 LaunchTask

```
override def receive: PartialFunction[Any, Unit] = {
    ... ...
    case LaunchTask(data) =>
      if (executor == null) {
        exitExecutor(1, "Received LaunchTask command but executor was null")
      } else {
        val taskDesc = TaskDescription.decode(data.value)
        logInfo("Got assigned task " + taskDesc.taskId)
        taskResources(taskDesc.taskId) = taskDesc.resources
        executor.launchTask(this, taskDesc)
      }
      ... ...
}
```

Executor.scala

```
def launchTask(context: ExecutorBackend, taskDescription: TaskDescription): Unit = {
    val tr = new TaskRunner(context, taskDescription)
    runningTasks.put(taskDescription.taskId, tr)
    threadPool.execute(tr)
}
```

第 3 章 Shuffle

Spark 最初版本 HashShuffle

Spark0.8.1 版本以后优化后的 HashShuffle

Spark1.1 版本加入 SortShuffle，默认是 HashShuffle

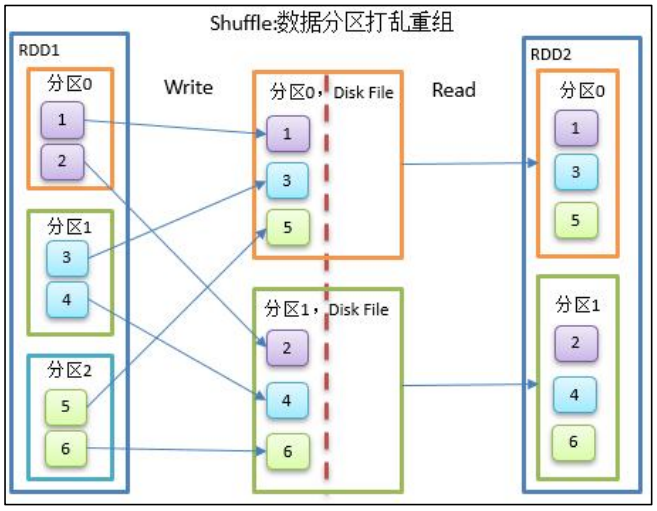
Spark1.2 版本默认是 SortShuffle，但是可配置 HashShuffle

Spark2.0 版本删除 HashShuffle 只有 SortShuffle

3.1 Shuffle 的原理和执行过程

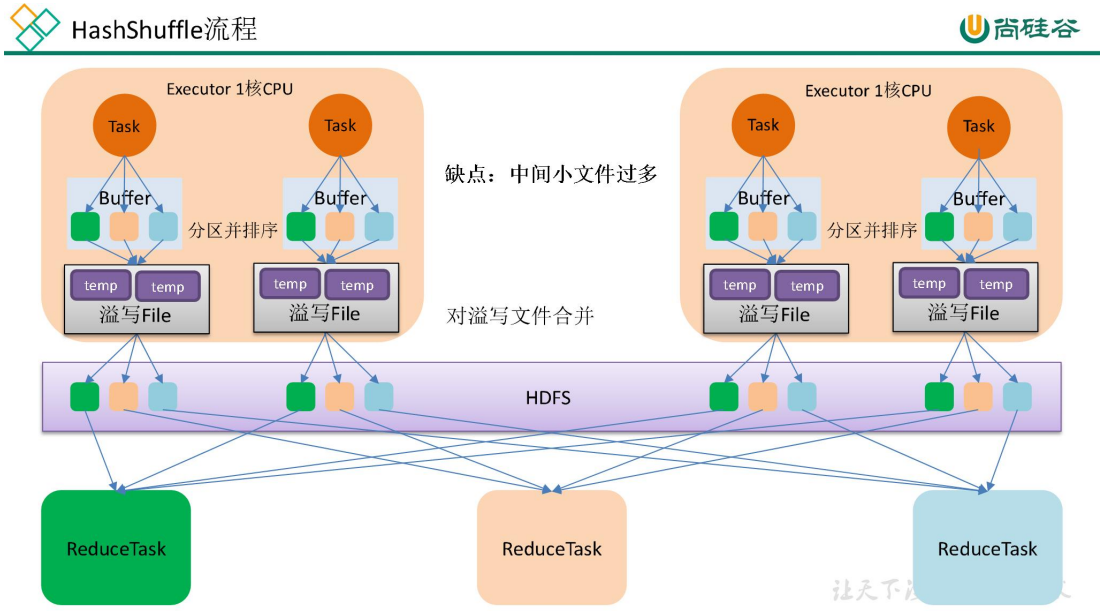
Shuffle 一定会有落盘。

- 如果 shuffle 过程中落盘数据量减少，那么可以提高性能。
- 算子如果存在预聚合功能，可以提高 shuffle 的性能。



3.2 HashShuffle 解析

3.2.1 未优化的 HashShuffle

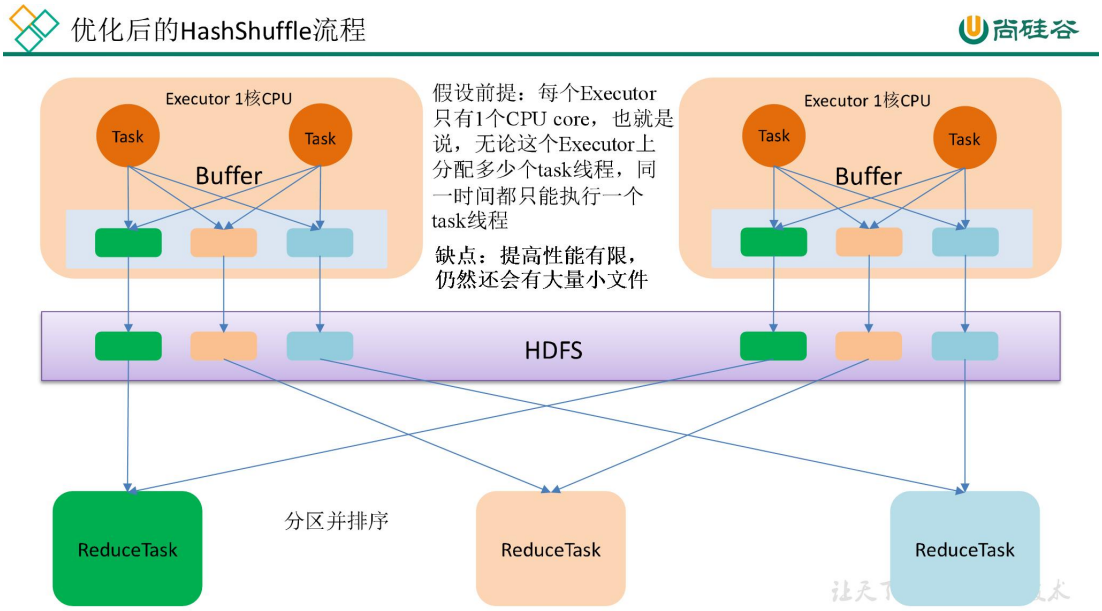


3.2.2 优化后的 HashShuffle

优化的 HashShuffle 过程就是启用合并机制，合并机制就是复用 buffer，开启合并机制

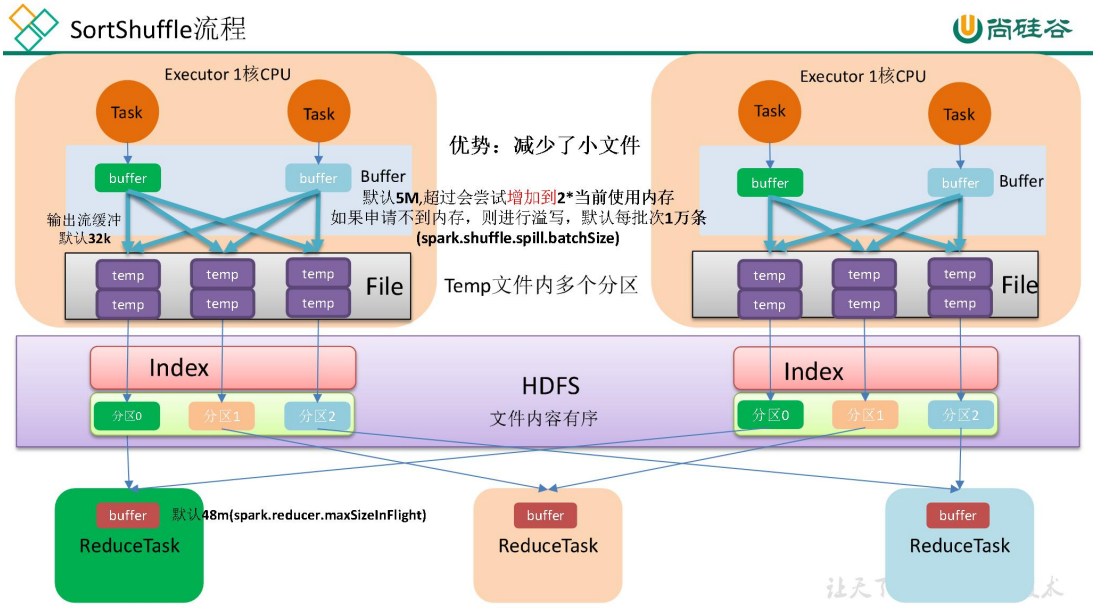
的配置是 spark.shuffle.consolidateFiles。该参数默认值为 false，将其设置为 true 即可开启优化机制。通常来说，如果我们使用 HashShuffleManager，那么都建议开启这个选项。

官网参数说明：http://spark.apache.org/docs/0.8.1/configuration.html



3.3 SortShuffle 解析

3.3.1 SortShuffle



在该模式下，数据会先写入一个数据结构，reduceByKey 写入 Map，一边通过 Map 局部聚合，一边写入内存。Join 算子写入 ArrayList 直接写入内存中。然后需要判断是否达到阈值，如果达到就会将内存数据结构的数据写入到磁盘，清空内存数据结构。

在溢写磁盘前，先根据 key 进行排序，排序过后的数据，会分批写入到磁盘文件中。默认批次为 10000 条，数据会以每批一万条写入到磁盘文件。写入磁盘文件通过缓冲区溢写的方式，每次溢写都会产生一个磁盘文件，也就是说一个 Task 过程会产生多个临时文件。最后在每个 Task 中，将所有的临时文件合并，这就是 merge 过程，此过程将所有临时文件读取出来，一次写入到最终文件。意味着一个 Task 的所有数据都在这一个文件中。同时单独写一份索引文件，标识下游各个 Task 的数据在文件中的索引，start offset 和 end offset。

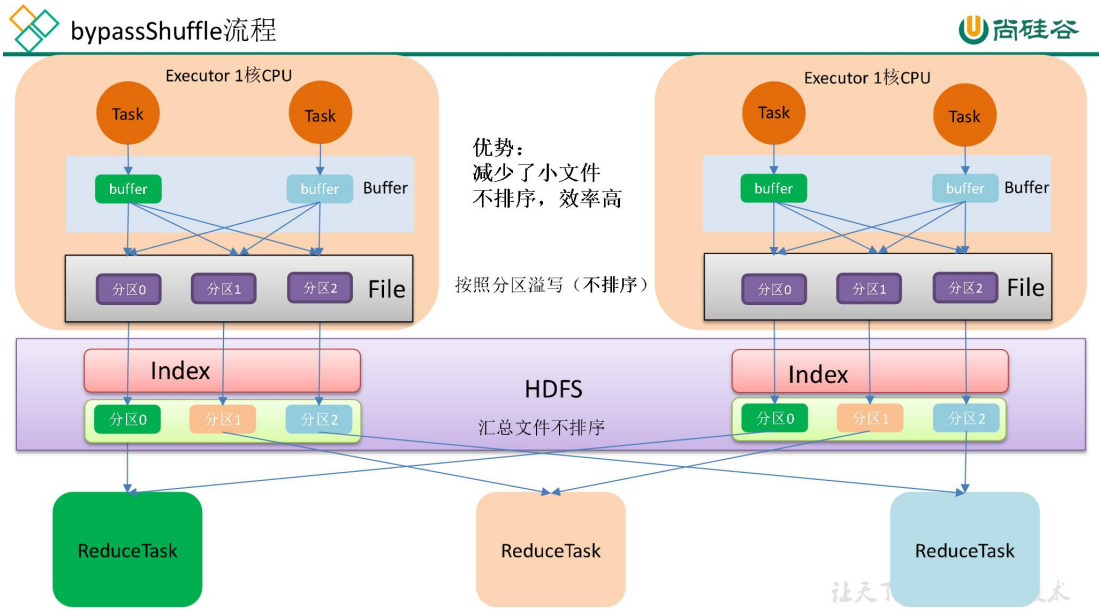
3.3.2 bypassShuffle

bypassShuffle 和 SortShuffle 的区别就是不对数据排序。

bypass 运行机制的触发条件如下：

1）shuffle reduce task 数量小于等于 spark.shuffle.sort.bypassMergeThreshold 参数的值，默认为 200。

2）不是聚合类的 shuffle 算子（比如 reduceByKey 不行）。



3.4 Shuffle 写磁盘

3.4.1 shuffleWriterProcessor（写处理器）

DAGScheduler.scala

```
private def submitMissingTasks(stage: Stage, jobId: Int): Unit = {
      ... ...
      val tasks: Seq[Task[_]] = try {
            val                                  serializedTaskMetrics                                  =
closureSerializer.serialize(stage.latestInfo.taskMetrics).array()
```

```
            stage match {
            // shuffle 写过程
            case stage: ShuffleMapStage =>
                stage.pendingPartitions.clear()
                partitionsToCompute.map { id =>
                val locs = taskIdToLocations(id)
                val part = partitions(id)
                stage.pendingPartitions += id
                new ShuffleMapTask(stage.id, stage.latestInfo.attemptNumber,
                    taskBinary, part, locs, properties, serializedTaskMetrics, Option(jobId),
                    Option(sc.applicationId), sc.applicationAttemptId, stage.rdd.isBarrier())
                }
            // shuffle 读过程
            case stage: ResultStage =>
            ... ...
            }
        } catch {
            ... ...
        }
}
```

Task.scala

```
private[spark] abstract class Task[T](... ...) extends Serializable {

    final def run(... ...): T = {
        runTask(context)
    }
}
```

Ctrl+h 查找 runTask 实现类 ShuffleMapTask.scala

```
private[spark] class ShuffleMapTask(... ...)
  extends Task[MapStatus](... ...){

    override def runTask(context: TaskContext): MapStatus = {
        ... ...
        dep.shuffleWriterProcessor.write(rdd, dep, mapId, context, partition)
    }
}
```

ShuffleWriteProcessor.scala

```
def write(... ...): MapStatus = {

    var writer: ShuffleWriter[Any, Any] = null
    try {
        val manager = SparkEnv.get.shuffleManager
        writer = manager.getWriter[Any, Any](
        dep.shuffleHandle,
        mapId,
        context,
        createMetricsReporter(context))

        writer.write(
        rdd.iterator(partition, context).asInstanceOf[Iterator[_ <: Product2[Any, Any]]])
        writer.stop(success = true).get
    } catch {
        ... ...
```

```
    }
}
```

查找（ctrl + h）ShuffleManager 的实现类，SortShuffleManager

SortShuffleManager.scala

```
override def getWriter[K, V](  handle: ShuffleHandle,
        mapId: Long,
        context: TaskContext,
        metrics: ShuffleWriteMetricsReporter): ShuffleWriter[K, V] =
    ... ...
    handle match {
    case unsafeShuffleHandle: SerializedShuffleHandle[K @unchecked, V @unchecked]
=>
        new UnsafeShuffleWriter(... ...)
    case bypassMergeSortHandle: BypassMergeSortShuffleHandle[K @unchecked, V
@unchecked] =>
        new BypassMergeSortShuffleWriter(... ...)
    case other: BaseShuffleHandle[K @unchecked, V @unchecked, _] =>
        new SortShuffleWriter(... ...)
    }
}
```

因为 getWriter 的第一个输入参数是 dep.shuffleHandle，点击 dep.shuffleHandle

Dependency.scala

```
val shuffleHandle: ShuffleHandle =
_rdd.context.env.shuffleManager.registerShuffle(shuffleId, this)
```

ShuffleManager.scala

```
def registerShuffle[K, V, C](shuffleId: Int, dependency: ShuffleDependency[K, V, C]):
ShuffleHandle
```

3.4.2 使用 BypassShuffle 条件

BypassMergeSortShuffleHandle 使用条件：

1）不能使用预聚合

2）如果下游的分区数量小于等于 200（可配置）

处理器	写对象	判断条件
SerializedShuffleHandle	UnsafeShuffleWriter	1.序列化规则支持重定位操作（java 序列化不支持，Kryo 支持） 2.不能使用预聚合 3.如果下游的分区数量小于或等于 1677216
BypassMergeSortShuffleHandle	BypassMergeSortShuffleWriter	1.不能使用预聚合 2.如果下游的分区数量小于等于 200

		（可配置）
BaseShuffleHandle	SortShuffleWriter	其他情况

查找（ctrl + h）registerShuffle 实现类，SortShuffleManager.scala

```
override def registerShuffle[K, V, C](
    shuffleId: Int,
    dependency: ShuffleDependency[K, V, C]): ShuffleHandle = {
    //使用 BypassShuffle 条件：不能使用预聚合功能；默认下游分区数据不能大于 200
    if (SortShuffleWriter.shouldBypassMergeSort(conf, dependency)) {
        new BypassMergeSortShuffleHandle[K, V](
            shuffleId, dependency.asInstanceOf[ShuffleDependency[K, V, V]])
    } else if (SortShuffleManager.canUseSerializedShuffle(dependency)) {
        new SerializedShuffleHandle[K, V](
            shuffleId, dependency.asInstanceOf[ShuffleDependency[K, V, V]])
    } else {
        new BaseShuffleHandle(shuffleId, dependency)
    }
}
```

点击 shouldBypassMergeSort

SortShuffleWriter.scala

```
private[spark] object SortShuffleWriter {
    def shouldBypassMergeSort(conf: SparkConf, dep: ShuffleDependency[ , , ]):
Boolean = {
        // 是否有 map 阶段预聚合（支持预聚合不能用）
        if (dep.mapSideCombine) {
            false
        } else {
// SHUFFLE_SORT_BYPASS_MERGE_THRESHOLD = 200 分区
            val bypassMergeThreshold: Int =
conf.get(config.SHUFFLE_SORT_BYPASS_MERGE_THRESHOLD)
            // 如果下游分区器的数量，小于 200（可配置），可以使用 bypass
            dep.partitioner.numPartitions <= bypassMergeThreshold
        }
    }
}
```

3.4.3 使用 SerializedShuffle 条件

SerializedShuffleHandle 使用条件：

1）序列化规则支持重定位操作（java 序列化不支持，Kryo 支持）

2）不能使用预聚合

3）如果下游的分区数量小于或等于 1677216

点击 canUseSerializedShuffle

SortShuffleManager.scala

```
def canUseSerializedShuffle(dependency: ShuffleDependency[_, _, _]): Boolean = {
    val shufId = dependency.shuffleId
    val numPartitions = dependency.partitioner.numPartitions
```

```
    // 是否支持将两个独立的序列化对象 重定位，聚合到一起
    // 1 默认的 java 序列化不支持；Kryo 序列化支持重定位（可以用）
    if (!dependency.serializer.supportsRelocationOfSerializedObjects) {
        false
    } else if (dependency.mapSideCombine) { // 2 支持预聚合也不能用
        false
    } else if (numPartitions > MAX_SHUFFLE_OUTPUT_PARTITIONS_FOR_SERIALIZED_MODE) {//3 如果下游分区的数量大于 16777216，也不能用
        false
    } else {
        true
    }
}
```

3.4.4 使用 BaseShuffle

点击 SortShuffleWriter

SortShuffleWriter.scala

```
override def write(records: Iterator[Product2[K, V]]): Unit = {

    // 判断是否有预聚合功能，支持会有 aggregator 和排序规则
    sorter = if (dep.mapSideCombine) {
        new ExternalSorter[K, V, C](
        context, dep.aggregator, Some(dep.partitioner), dep.keyOrdering, dep.serializer)
    } else {
        new ExternalSorter[K, V, V](
        context, aggregator = None, Some(dep.partitioner), ordering = None, dep.serializer)
    }
    // 插入数据
    sorter.insertAll(records)

    val mapOutputWriter = shuffleExecutorComponents.createMapOutputWriter(
        dep.shuffleId, mapId, dep.partitioner.numPartitions)
    // 插入数据
    sorter.writePartitionedMapOutput(dep.shuffleId, mapId, mapOutputWriter)

    val partitionLengths = mapOutputWriter.commitAllPartitions()

    mapStatus = MapStatus(blockManager.shuffleServerId, partitionLengths, mapId)
}
```

3.4.5 插入数据（缓存+溢写）

ExternalSorter.scala

```
def insertAll(records: Iterator[Product2[K, V]]): Unit = {

    val shouldCombine = aggregator.isDefined

    // 判断是否支持预聚合，支持预聚合，采用 map 结构，不支持预聚合采用 buffer 结构
```

```
        if (shouldCombine) {
            val mergeValue = aggregator.get.mergeValue
            val createCombiner = aggregator.get.createCombiner
            var kv: Product2[K, V] = null
            val update = (hadValue: Boolean, oldValue: C) => {
                if (hadValue) mergeValue(oldValue, kv._2) else createCombiner(kv._2)
            }

            while (records.hasNext) {
                addElementsRead()
                kv = records.next()
                // 如果支持预聚合，在 map 阶段聚合，将相同 key，的 value 聚合
                map.changeValue((getPartition(kv._1), kv._1), update)
                // 是否能够溢写
                maybeSpillCollection(usingMap = true)
            }
        } else {
            while (records.hasNext) {
                addElementsRead()
                val kv = records.next()
                // 如果不支持预聚合，value 不需要聚合 (key，(value1,value2))
                buffer.insert(getPartition(kv._1), kv._1, kv._2.asInstanceOf[C])
                maybeSpillCollection(usingMap = false)
            }
        }
    }

    private def maybeSpillCollection(usingMap: Boolean): Unit = {
        var estimatedSize = 0L
        if (usingMap) {
            estimatedSize = map.estimateSize()
            if (maybeSpill(map, estimatedSize)) {
                map = new PartitionedAppendOnlyMap[K, C]
            }
        } else {
            estimatedSize = buffer.estimateSize()
            if (maybeSpill(buffer, estimatedSize)) {
                buffer = new PartitionedPairBuffer[K, C]
            }
        }

        if (estimatedSize > _peakMemoryUsedBytes) {
            _peakMemoryUsedBytes = estimatedSize
        }
    }
```

Spillable.scala

```
protected def maybeSpill(collection: C, currentMemory: Long): Boolean = {
    var shouldSpill = false
    // myMemoryThreshold 默认值内存门槛是 5m
    if (elementsRead % 32 == 0 && currentMemory >= myMemoryThreshold) {

        val amountToRequest = 2 * currentMemory - myMemoryThreshold
        // 申请内存
        val granted = acquireMemory(amountToRequest)
```

```
            myMemoryThreshold += granted
            // 当前内存大于（尝试申请的内存+门槛），就需要溢写了
            shouldSpill = currentMemory >= myMemoryThreshold
        }
        // 强制溢写 读取数据的值 超过了 Int 的最大值
        shouldSpill = shouldSpill || _elementsRead > numElementsForceSpillThreshold

        if (shouldSpill) {
            _spillCount += 1
            logSpillage(currentMemory)
            // 溢写
            spill(collection)
            _elementsRead = 0
            _memoryBytesSpilled += currentMemory
            // 释放内存
            releaseMemory()
        }
        shouldSpill
}

protected def spill(collection: C): Unit
```

查找（ctrl +h）spill 的实现类 ExternalSorter

ExternalSorter.scala

```
override protected[this] def spill(collection: WritablePartitionedPairCollection[K, C]): Unit =
{
    val inMemoryIterator =
collection.destructiveSortedWritablePartitionedIterator(comparator)
    val spillFile = spillMemoryIteratorToDisk(inMemoryIterator)
    spills += spillFile
}

private[this] def spillMemoryIteratorToDisk(inMemoryIterator: WritablePartitionedIterator)
      : SpilledFile = {
  // 创建临时文件
  val (blockId, file) = diskBlockManager.createTempShuffleBlock()

  var objectsWritten: Long = 0
  val spillMetrics: ShuffleWriteMetrics = new ShuffleWriteMetrics
  // 溢写文件前，fileBufferSize 缓冲区大小默认 32m
  val writer: DiskBlockObjectWriter =
    blockManager.getDiskWriter(blockId, file, serInstance, fileBufferSize, spillMetrics)

  … …

  SpilledFile(file, blockId, batchSizes.toArray, elementsPerPartition)
}
```

3.4.6 merge 合并

来到 SortShuffleWriter.scala

```
override def write(records: Iterator[Product2[K, V]]): Unit = {
    sorter = if (dep.mapSideCombine) {
        new ExternalSorter[K, V, C](
```

```
            context, dep.aggregator, Some(dep.partitioner), dep.keyOrdering, dep.serializer)
        } else {
            new ExternalSorter[K, V, V](
            context, aggregator = None, Some(dep.partitioner), ordering = None,
dep.serializer)
        }
        sorter.insertAll(records)

        val mapOutputWriter = shuffleExecutorComponents.createMapOutputWriter(
            dep.shuffleId, mapId, dep.partitioner.numPartitions)
        // 合并
        sorter.writePartitionedMapOutput(dep.shuffleId, mapId, mapOutputWriter)

        val partitionLengths = mapOutputWriter.commitAllPartitions()

        mapStatus = MapStatus(blockManager.shuffleServerId, partitionLengths, mapId)
}
```

ExternalSorter.scala

```
def writePartitionedMapOutput(
        shuffleId: Int,
        mapId: Long,
        mapOutputWriter: ShuffleMapOutputWriter): Unit = {
        var nextPartitionId = 0
        // 如果溢写文件为空，只对内存中数据处理
        if (spills.isEmpty) {
            // Case where we only have in-memory data
            ... ...
        } else {
            // We must perform merge-sort; get an iterator by partition and write everything
directly.
            //如果溢写文件不为空，需要将多个溢写文件合并
            for ((id, elements) <- this.partitionedIterator) {
                val blockId = ShuffleBlockId(shuffleId, mapId, id)
                var partitionWriter: ShufflePartitionWriter = null
                var partitionPairsWriter: ShufflePartitionPairsWriter = null
                … …
                } {
                    if (partitionPairsWriter != null) {
                        partitionPairsWriter.close()
                    }
                }
                nextPartitionId = id + 1
            }
        }
        … …
}

def partitionedIterator: Iterator[(Int, Iterator[Product2[K, C]])] = {
        val usingMap = aggregator.isDefined
        val collection: WritablePartitionedPairCollection[K, C] = if (usingMap) map else buffer

        if (spills.isEmpty) {
            if (ordering.isEmpty) {

groupByPartition(destructiveIterator(collection.partitionedDestructiveSortedIterator(None)))
```

```
            } else {
                groupByPartition(destructiveIterator(
                collection.partitionedDestructiveSortedIterator(Some(keyComparator))))
            }
        } else {
            // 合并溢写文件和内存中数据
            merge(spills, destructiveIterator(
            collection.partitionedDestructiveSortedIterator(comparator)))
        }
}

private def merge(spills: Seq[SpilledFile], inMemory: Iterator[((Int, K), C)])
    : Iterator[(Int, Iterator[Product2[K, C]])] = {

    val readers = spills.map(new SpillReader(_))
    val inMemBuffered = inMemory.buffered

    (0 until numPartitions).iterator.map { p =>
        val inMemIterator = new IteratorForPartition(p, inMemBuffered)
        val iterators = readers.map(_.readNextPartition()) ++ Seq(inMemIterator)
        if (aggregator.isDefined) {

            (p, mergeWithAggregation(
            iterators,          aggregator.get.mergeCombiners,          keyComparator,
ordering.isDefined))
        } else if (ordering.isDefined) {
        // 归并排序
            (p, mergeSort(iterators, ordering.get))
        } else {
            (p, iterators.iterator.flatten)
        }
    }
}
```

3.4.7 写磁盘

来到 SortShuffleWriter.scala

```
override def write(records: Iterator[Product2[K, V]]): Unit = {
    sorter = if (dep.mapSideCombine) {
        new ExternalSorter[K, V, C](
        context, dep.aggregator, Some(dep.partitioner), dep.keyOrdering, dep.serializer)
    } else {
        new ExternalSorter[K, V, V](
        context,  aggregator  =  None,  Some(dep.partitioner),  ordering  =  None,
dep.serializer)
    }
    sorter.insertAll(records)

    val mapOutputWriter = shuffleExecutorComponents.createMapOutputWriter(
        dep.shuffleId, mapId, dep.partitioner.numPartitions)
    // 合并
    sorter.writePartitionedMapOutput(dep.shuffleId, mapId, mapOutputWriter)
    // 写磁盘
    val partitionLengths = mapOutputWriter.commitAllPartitions()
```

```
        mapStatus = MapStatus(blockManager.shuffleServerId, partitionLengths, mapId)
}
```

查找（ctrl + h）commitAllPartitions 实现类，来到 LocalDiskShuffleMapOutputWriter.java

```
public long[] commitAllPartitions() throws IOException {

        if (outputFileChannel != null && outputFileChannel.position() != bytesWrittenToMergedFile) {
                ... ...
        }
        cleanUp();
        File resolvedTmp = outputTempFile != null && outputTempFile.isFile() ? outputTempFile : null;
        blockResolver.writeIndexFileAndCommit(shuffleId, mapId, partitionLengths, resolvedTmp);
        return partitionLengths;
}
```

IndexShuffleBlockResolver.scala

```
def writeIndexFileAndCommit(
        shuffleId: Int,
        mapId: Long,
        lengths: Array[Long],
        dataTmp: File): Unit = {

        val indexFile = getIndexFile(shuffleId, mapId)
        val indexTmp = Utils.tempFileWith(indexFile)
        try {
                val dataFile = getDataFile(shuffleId, mapId)

                synchronized {
                        val existingLengths = checkIndexAndDataFile(indexFile, dataFile, lengths.length)
                        if (existingLengths != null) {
                                System.arraycopy(existingLengths, 0, lengths, 0, lengths.length)
                                if (dataTmp != null && dataTmp.exists()) {
                                        dataTmp.delete()
                                }
                        } else {
                                val out = new DataOutputStream(new BufferedOutputStream(new FileOutputStream(indexTmp)))
                                Utils.tryWithSafeFinally {
                                        var offset = 0L
                                        out.writeLong(offset)
                                        for (length <- lengths) {
                                                offset += length
                                                out.writeLong(offset)
                                        }
                                } {
                                        out.close()
                                }

                                if (indexFile.exists()) {
                                        indexFile.delete()
                                }
                                if (dataFile.exists()) {
```

```
                        dataFile.delete()
                    }
                    if (!indexTmp.renameTo(indexFile)) {
                        throw new IOException("fail to rename file " + indexTmp + " to " +
indexFile)
                    }
                    if (dataTmp != null && dataTmp.exists()
&& !dataTmp.renameTo(dataFile)) {
                        throw new IOException("fail to rename file " + dataTmp + " to " +
dataFile)
                    }
                }
            }
        } finally {
            ... ...
        }
}
```

3.5 Shuffle 读取磁盘

DAGScheduler.scala

```
private def submitMissingTasks(stage: Stage, jobId: Int): Unit = {
    ... ...
    val tasks: Seq[Task[ ]] = try {
        val serializedTaskMetrics =
closureSerializer.serialize(stage.latestInfo.taskMetrics).array()

        stage match {
        case stage: ShuffleMapStage =>
        ... ...
        case stage: ResultStage =>
            partitionsToCompute.map { id =>
            val p: Int = stage.partitions(id)
            val part = partitions(p)
            val locs = taskIdToLocations(id)
            new ResultTask(stage.id, stage.latestInfo.attemptNumber,
                taskBinary, part, locs, id, properties, serializedTaskMetrics,
                Option(jobId), Option(sc.applicationId), sc.applicationAttemptId,
                stage.rdd.isBarrier())
            }
        }
    } catch {
        ... ...
    }
}
```

ResultTask.scala

```
private[spark] class ResultTask[T, U](... ...)
  extends Task[U](... ...)
  with Serializable {

    override def runTask(context: TaskContext): U = {
        func(context, rdd.iterator(partition, context))
    }
}
```

RDD.scala

```
final def iterator(split: Partition, context: TaskContext): Iterator[T] = {
    if (storageLevel != StorageLevel.NONE) {
        getOrCompute(split, context)
    } else {
        computeOrReadCheckpoint(split, context)
    }
}

private[spark] def getOrCompute(partition: Partition, context: TaskContext): Iterator[T] = {
    ... ...
    computeOrReadCheckpoint(partition, context)
    ... ...
}

def computeOrReadCheckpoint(split: Partition, context: TaskContext): Iterator[T] ={
    if (isCheckpointedAndMaterialized) {
        firstParent[T].iterator(split, context)
    } else {
        compute(split, context)
    }
}

def compute(split: Partition, context: TaskContext): Iterator[T]
```

全局查找 compute，由于我们是 ShuffledRDD，所以点击 ShuffledRDD.scala，搜索 compute

```
override def compute(split: Partition, context: TaskContext): Iterator[(K, C)] = {
    val dep = dependencies.head.asInstanceOf[ShuffleDependency[K, V, C]]
    val metrics = context.taskMetrics().createTempShuffleReadMetrics()
    SparkEnv.get.shuffleManager.getReader(
    dep.shuffleHandle, split.index, split.index + 1, context, metrics)
    .read()
    .asInstanceOf[Iterator[(K, C)]]
}
```

ShuffleManager.scala 文件

```
def getReader[K, C](... ...): ShuffleReader[K, C]
```

查找（ctrl + h）getReader 的实现类，SortShuffleManager.scala

```
override def getReader[K, C](... ...): ShuffleReader[K, C] = {
    val blocksByAddress = SparkEnv.get.mapOutputTracker.getMapSizesByExecutorId(
        handle.shuffleId, startPartition, endPartition)
    new BlockStoreShuffleReader(
        handle.asInstanceOf[BaseShuffleHandle[K, _, C]], blocksByAddress, context,
metrics,
        shouldBatchFetch = canUseBatchFetch(startPartition, endPartition, context))
}
```

在 BlockStoreShuffleReader.scala 文件中查找 read 方法

```
override def read(): Iterator[Product2[K, C]] = {
    val wrappedStreams = new ShuffleBlockFetcherIterator(
        ... ...
        // 读缓冲区大小 默认 48m
        SparkEnv.get.conf.get(config.REDUCER_MAX_SIZE_IN_FLIGHT) * 1024 *
1024,
```

```
            SparkEnv.get.conf.get(config.REDUCER_MAX_REQS_IN_FLIGHT),
            ... ...
}
```

第 4 章 Spark 内存管理

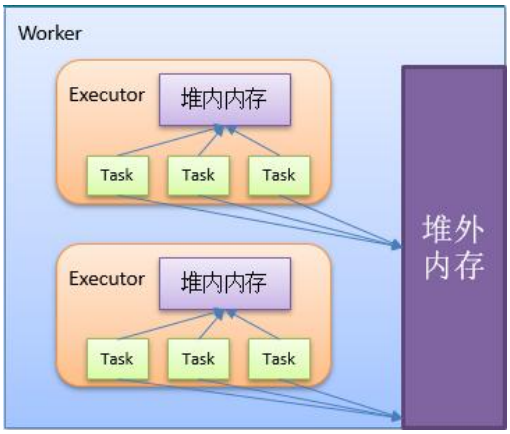
4.1 堆内和堆外内存

4.1.1 概念

Spark 支持堆内内存也支持堆外内存

1）堆内内存：程序在运行时动态地申请某个大小的内存空间

2）堆外内存：直接向操作系统进行申请的内存，不受 JVM 控制



4.1.2 堆内内存和对外内存优缺点

1）堆外内存，相比于堆内内存有几个优势：

（1）减少了垃圾回收的工作，因为垃圾回收会暂停其他的工作

（2）加快了复制的速度。因为堆内在 Flush 到远程时，会先序列化，然后在发送；而堆外内存本身是序列化的相当于省略掉了这个工作。

说明：堆外内存是序列化的，其占用的内存大小可直接计算。堆内内存是非序列化的对象，其占用的内存是通过周期性地采样近似估算而得，即并不是每次新增的数据项都会计算一次占用的内存大小，这种方法降低了时间开销但是有可能误差较大，导致某一时刻的实际内存有可能远远超出预期。此外，在被 Spark 标记为释放的对象实例，很有可能在实际上并没有被 JVM 回收，导致实际可用的内存小于 Spark 记录的可用内存。所以 Spark 并不能准确记录实际可用的堆内内存，从而也就无法完全避免内存溢出 OOM 的异常。

2）堆外内存，相比于堆内内存有几个缺点：

（1）堆外内存难以控制，如果内存泄漏，那么很难排查

（2）堆外内存相对来说，不适合存储很复杂的对象。一般简单的对象或者扁平化的比较适合。

4.1.3 如何配置

1）堆内内存大小设置：–executor-memory 或 spark.executor.memory

2）在默认情况下堆外内存并不启用，spark.memory.offHeap.enabled 参数启用，并由 spark.memory.offHeap.size 参数设定堆外空间的大小。

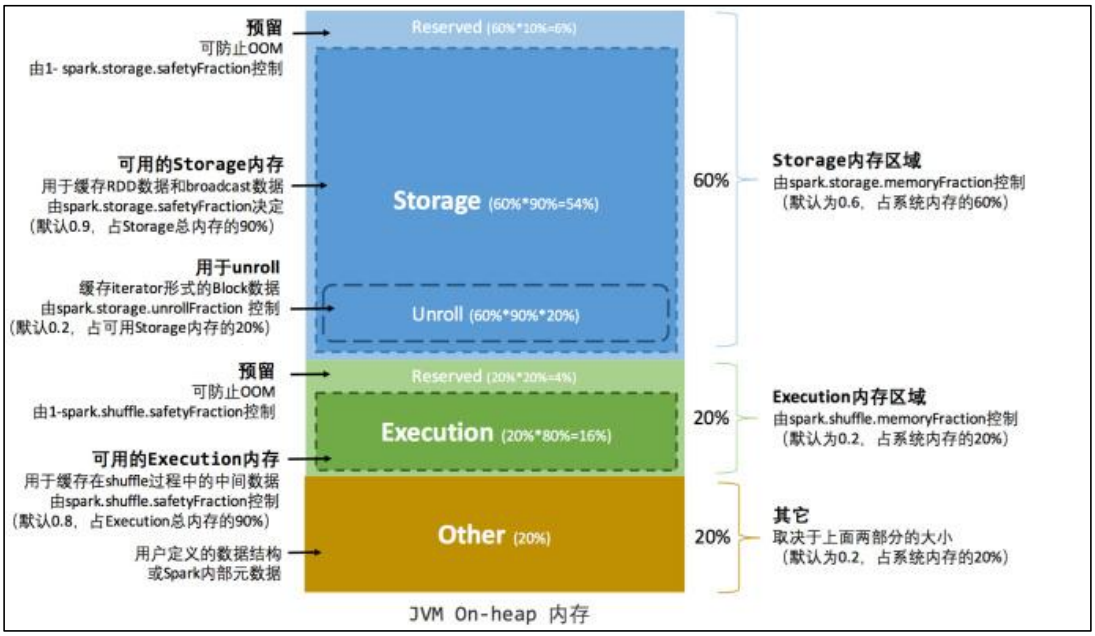
官网配置地址：http://spark.apache.org/docs/3.0.0/configuration.html

4.2 堆内内存空间分配

堆内内存包括：**存储（Storage）内存、执行（Execution）内存、其他内存**

4.2.1 静态内存管理

在 **Spark 最初采用的静态内存管理机制**下，存储内存、执行内存和其他内存的大小在 Spark 应用程序运行期间均为固定的，但用户可以应用程序启动前进行配置，堆内内存的分配如图所示：



可以看到，可用的堆内内存的大小需要按照下列方式计算：

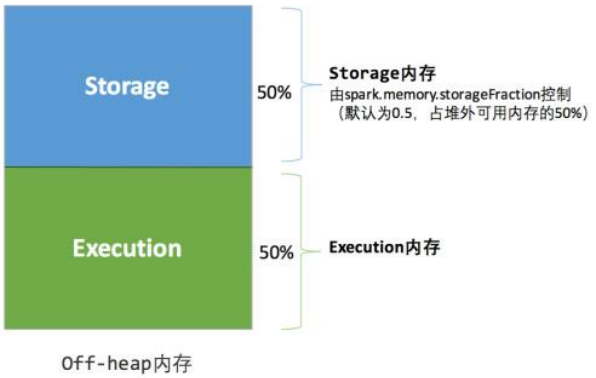
```
可用的存储内存 = systemMaxMemory * spark.storage.memoryFraction * spark.storage.safetyFraction
可用的执行内存 = systemMaxMemory * spark.shuffle.memoryFraction * spark.shuffle.safetyFraction
```

其中 systemMaxMemory 取决于当前 JVM 堆内内存的大小，最后可用的执行内存或者存储内存要在此基础上与各自的 memoryFraction 参数和 safetyFraction 参数相乘得出。上述计算

公式中的两个 safetyFraction 参数，其意义在于在逻辑上预留出 1-safetyFraction 这么一块保险区域，降低因实际内存超出当前预设范围而导致 OOM 的风险（上文提到，对于非序列化对象的内存采样估算会产生误差）。值得注意的是，这个预留的保险区域仅仅是一种逻辑上的规划，在具体使用时 Spark 并没有区别对待，和”其它内存”一样交给了 JVM 去管理。

Storage 内存和 Execution 内存都有预留空间，目的是防止 OOM，因为 Spark 堆内内存大小的记录是不准确的，需要留出保险区域。

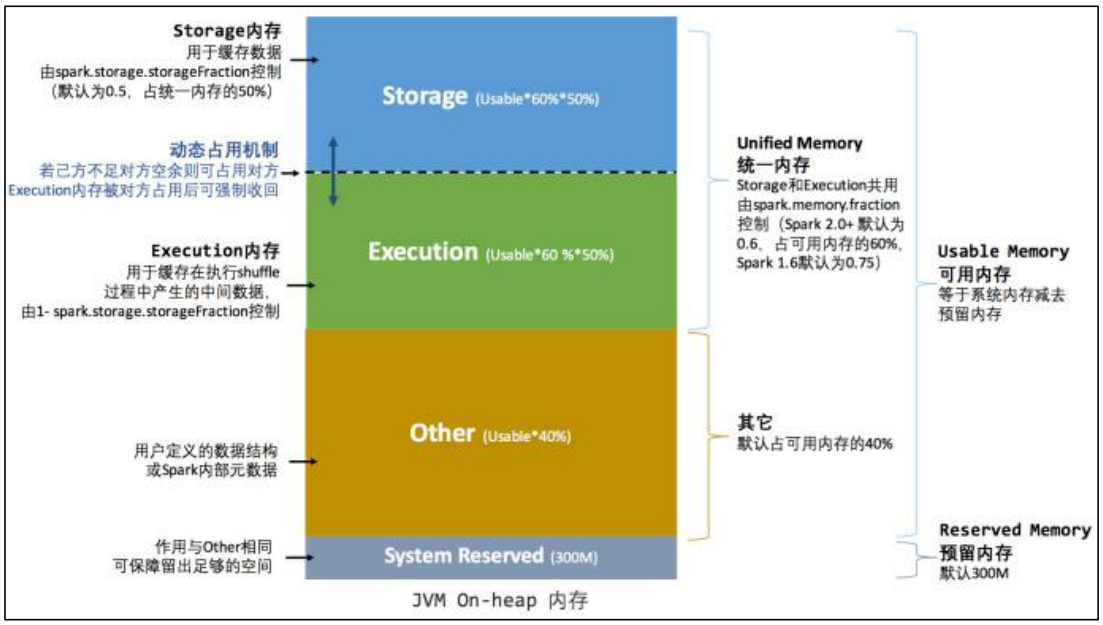
堆外的空间分配较为简单，只有存储内存和执行内存，如下图所示。可用的执行内存和存储内存占用的空间大小直接由参数 spark.memory.storageFraction 决定，由于堆外内存占用的空间可以被精确计算，所以无需再设定保险区域。



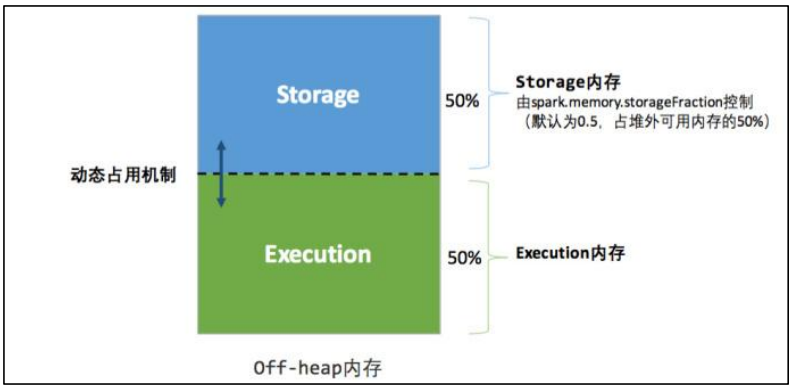
静态内存管理机制实现起来较为简单，但如果用户不熟悉 Spark 的存储机制，或没有根据具体的数据规模和计算任务或做相应的配置，很容易造成”一半海水，一半火焰”的局面，即存储内存和执行内存中的一方剩余大量的空间，而另一方却早早被占满，不得不淘汰或移出旧的内容以存储新的内容。由于新的内存管理机制的出现，这种方式目前已经很少有开发者使用，出于兼容旧版本的应用程序的目的，Spark 仍然保留了它的实现。

4.2.2 统一内存管理

Spark1.6 之后引入的统一内存管理机制，与静态内存管理的区别在于存储内存和执行内存共享同一块空间，可以动态占用对方的空闲区域，统一内存管理的堆内内存结构如图所示：



统一内存管理的堆外内存结构如下图所示：



其中最重要的优化在于动态占用机制，其规则如下：

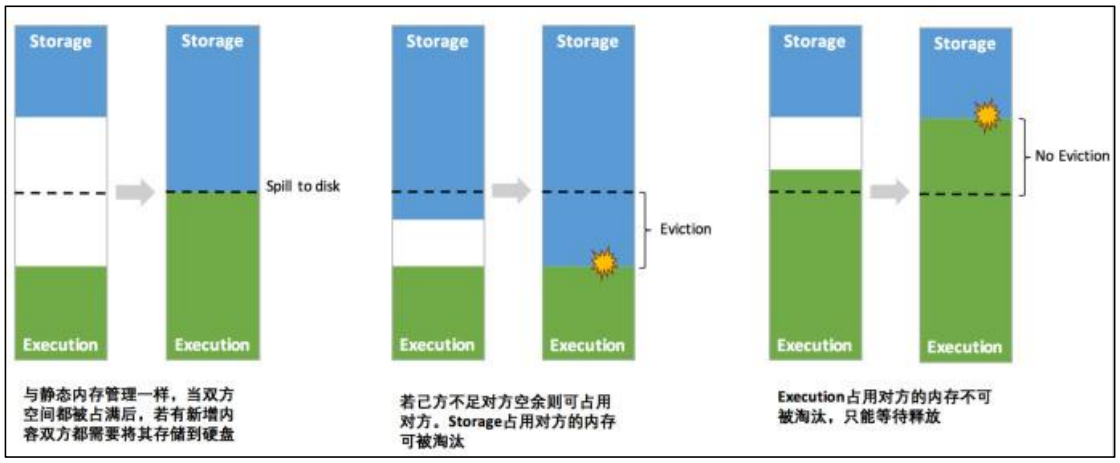
1) 设定基本的存储内存和执行内存区域（spark.storage.storageFraction 参数），该设定确定了双方各自拥有的空间的范围；

2) 双方的空间都不足时，则存储到硬盘；若己方空间不足而对方空余时，可借用对方的空间;（存储空间不足是指不足以放下一个完整的 Block）

3) 执行内存的空间被对方占用后，可让对方将占用的部分转存到硬盘，然后”归还”借用的空间；

4) 存储内存的空间被对方占用后，无法让对方”归还”，因为需要考虑 Shuffle 过程中的很多因素，实现起来较为复杂。

统一内存管理的动态占用机制如图所示：



凭借统一内存管理机制，Spark 在一定程度上提高了堆内和堆外内存资源的利用率，降低了开发者维护 Spark 内存的难度，但并不意味着开发者可以高枕无忧。如果存储内存的空间太大或者说缓存的数据过多，反而会导致频繁的全量垃圾回收，降低任务执行时的性能，因为缓存的 RDD 数据通常都是长期驻留内存的。所以要想充分发挥 Spark 的性能，需要开发者进一步了解存储内存和执行内存各自的管理方式和实现原理。

4.2.3 内存空间分配

全局查找（ctrl + n）SparkEnv，并找到 create 方法

SparkEnv.scala

```
private def create(
      conf: SparkConf,
      executorId: String,
      bindAddress: String,
      advertiseAddress: String,
      port: Option[Int],
      isLocal: Boolean,
      numUsableCores: Int,
      ioEncryptionKey: Option[Array[Byte]],
      listenerBus: LiveListenerBus = null,
      mockOutputCommitCoordinator: Option[OutputCommitCoordinator] = None):
SparkEnv = {

      ... ...
      val memoryManager: MemoryManager = UnifiedMemoryManager(conf,
numUsableCores)
      ... ...
}
```

UnifiedMemoryManager.scala

```
def apply(conf: SparkConf, numCores: Int): UnifiedMemoryManager = {
      // 获取最大的可用内存为总内存的 0.6
      val maxMemory = getMaxMemory(conf)
      // 最大可用内存的 0.5 MEMORY_STORAGE_FRACTION=0.5
      new UnifiedMemoryManager(
            conf,
```

```
            maxHeapMemory = maxMemory,
            onHeapStorageRegionSize =
            (maxMemory * conf.get(config.MEMORY_STORAGE_FRACTION)).toLong,
            numCores = numCores)
}

private def getMaxMemory(conf: SparkConf): Long = {
    // 获取系统内存
    val systemMemory = conf.get(TEST_MEMORY)

    // 获取系统预留内存，默认 300m（RESERVED_SYSTEM_MEMORY_BYTES = 300
* 1024 * 1024）
    val reservedMemory = conf.getLong(TEST_RESERVED_MEMORY.key,
        if (conf.contains(IS_TESTING)) 0 else
RESERVED_SYSTEM_MEMORY_BYTES)
    val minSystemMemory = (reservedMemory * 1.5).ceil.toLong

    if (systemMemory < minSystemMemory) {
        throw new IllegalArgumentException(s"System memory $systemMemory must " +
        s"be at least $minSystemMemory. Please increase heap size using the
--driver-memory " +
        s"option or ${config.DRIVER_MEMORY.key} in Spark configuration.")
    }

    if (conf.contains(config.EXECUTOR_MEMORY)) {
        val executorMemory = conf.getSizeAsBytes(config.EXECUTOR_MEMORY.key)
        if (executorMemory < minSystemMemory) {
        throw new IllegalArgumentException(s"Executor memory $executorMemory must
be at least " +
            s"$minSystemMemory. Please increase executor memory using the " +
            s"--executor-memory option or ${config.EXECUTOR_MEMORY.key} in
Spark configuration.")
        }
    }
    val usableMemory = systemMemory - reservedMemory
    val memoryFraction = conf.get(config.MEMORY_FRACTION)
    (usableMemory * memoryFraction).toLong
}
```

config\package.scala

```
private[spark] val MEMORY_FRACTION = ConfigBuilder("spark.memory.fraction")
    ... ...
    .createWithDefault(0.6)
```

点击 UnifiedMemoryManager.apply()中的 UnifiedMemoryManager

```
private[spark] class UnifiedMemoryManager(
    conf: SparkConf,
    val maxHeapMemory: Long,
    onHeapStorageRegionSize: Long,
    numCores: Int)
  extends MemoryManager(
    conf,
    numCores,
    onHeapStorageRegionSize,
    maxHeapMemory - onHeapStorageRegionSize) {// 执行内存 0.6 -0.3 = 0.3
```

```
}
```

点击 MemoryManager

MemoryManager.scala

```
private[spark] abstract class MemoryManager(
    conf: SparkConf,
    numCores: Int,
    onHeapStorageMemory: Long,
    onHeapExecutionMemory: Long) extends Logging {// 执行内存 0.6 -0.3 = 0.3
    ... ...
    // 堆内存储内存
    protected val onHeapStorageMemoryPool = new StorageMemoryPool(this,
MemoryMode.ON_HEAP)
    // 堆外存储内存
    protected val offHeapStorageMemoryPool = new StorageMemoryPool(this,
MemoryMode.OFF_HEAP)
    // 堆内执行内存
    protected val onHeapExecutionMemoryPool = new ExecutionMemoryPool(this,
MemoryMode.ON_HEAP)
    // 堆外执行内存
    protected val offHeapExecutionMemoryPool = new ExecutionMemoryPool(this,
MemoryMode.OFF_HEAP)

    protected[this] val maxOffHeapMemory = conf.get(MEMORY_OFFHEAP_SIZE)
    // 堆外内存 MEMORY_STORAGE_FRACTION = 0.5
    protected[this] val offHeapStorageMemory =
        (maxOffHeapMemory * conf.get(MEMORY_STORAGE_FRACTION)).toLong
    ... ...
}
```

4.3 存储内存管理

4.3.1 RDD 的持久化机制

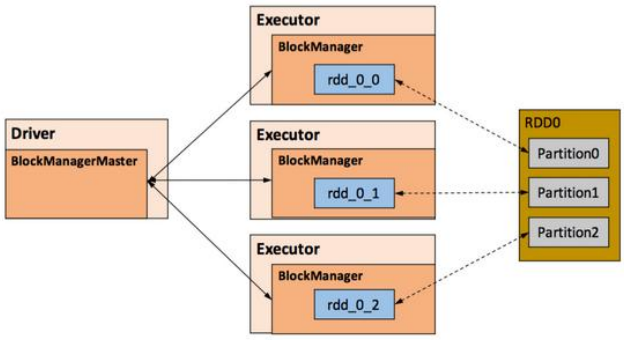
弹性分布式数据集（RDD）作为 Spark 最根本的数据抽象，是只读的分区记录（Partition）的集合，只能基于在稳定物理存储中的数据集上创建，或者在其他已有的 RDD 上执行转换（Transformation）操作产生一个新的 RDD。转换后的 RDD 与原始的 RDD 之间产生的依赖关系，构成了血统（Lineage）。凭借血统，Spark 保证了每一个 RDD 都可以被重新恢复。但 RDD 的所有转换都是惰性的，即只有当一个返回结果给 Driver 的行动（Action）发生时，Spark 才会创建任务读取 RDD，然后真正触发转换的执行。

Task 在启动之初读取一个分区时，会先判断这个分区是否已经被持久化，如果没有则需要检查 Checkpoint 或按照血统重新计算。所以如果一个 RDD 上要执行多次行动，可以在第一次行动中使用 persist 或 cache 方法，在内存或磁盘中持久化或缓存这个 RDD，从而在后面的行动时提升计算速度。

事实上，cache 方法是使用默认的 MEMORY_ONLY 的存储级别将 RDD 持久化到内

存，故缓存是一种特殊的持久化。 堆内和堆外存储内存的设计，便可以对缓存 RDD 时使用的内存做统一的规划和管理。

RDD 的持久化由 Spark 的 Storage 模块负责，实现了 RDD 与物理存储的解耦合。Storage 模块负责管理 Spark 在计算过程中产生的数据，将那些在内存或磁盘、在本地或远程存取数据的功能封装了起来。在具体实现时 Driver 端和 Executor 端的 Storage 模块构成了主从式的架构，即 Driver 端的 BlockManager 为 Master，Executor 端的 BlockManager 为 Slave。Storage 模块在逻辑上以 Block 为基本存储单位，RDD 的每个 Partition 经过处理后唯一对应一个 Block（BlockId 的格式为 rdd_RDD-ID_PARTITION-ID ）。Driver 端的 Master 负责整个 Spark 应用程序的 Block 的元数据信息的管理和维护，而 Executor 端的 Slave 需要将 Block 的更新等状态上报到 Master，同时接收 Master 的命令，例如新增或删除一个 RDD。



在对 RDD 持久化时，Spark 规定了 MEMORY_ONLY、MEMORY_AND_DISK 等 7 种不同的存储级别，而存储级别是以下 5 个变量的组合：

```
class StorageLevel private(
private var _useDisk: Boolean, //磁盘
private var _useMemory: Boolean, //这里其实是指堆内内存
private var _useOffHeap: Boolean, //堆外内存
private var _deserialized: Boolean, //是否为非序列化
private var _replication: Int = 1 //副本个数
)
```

Spark 中 7 种存储级别如下：

持久化级别	含义
MEMORY_ONLY	以非序列化的 Java 对象的方式持久化在 JVM 内存中。如果内存无法完全存储 RDD 所有的 partition，那么那些没有持久化的 partition 就会在下一次需要使用它们的时候，重新被计算
MEMORY_AND_DISK	同上，但是当某些 partition 无法存储在内存中时，会持久化到磁盘中。下次需要使用这些 partition 时，需要从磁盘上读取
MEMORY_ONLY_SER	同 MEMORY_ONLY，但是会使用 Java 序列化方式，将 Java 对象序列化后进行持久化。可以减少内存开销，但

	是需要进行反序列化，因此会加大 CPU 开销
MEMORY_AND_DISK_SER	同 MEMORY_AND_DISK，但是使用序列化方式持久化 Java 对象
DISK_ONLY	使用非序列化 Java 对象的方式持久化，完全存储到磁盘上
MEMORY_ONLY_2 MEMORY_AND_DISK_2 等等	如果是尾部加了 2 的持久化级别，表示将持久化数据复用一份，保存到其他节点，从而在数据丢失时，不需要再次计算，只需要使用备份数据即可

通过对数据结构的分析，可以看出存储级别从三个维度定义了 RDD 的 Partition（同时也就是 Block）的存储方式：

- **存储位置**：磁盘 / 堆内内存 / 堆外内存。如 MEMORY_AND_DISK 是同时在磁盘和堆内内存上存储，实现了冗余备份。OFF_HEAP 则是只在堆外内存存储，目前选择堆外内存时不能同时存储到其他位置。
- **存储形式**：Block 缓存到存储内存后，是否为非序列化的形式。如 MEMORY_ONLY 是非序列化方式存储，OFF_HEAP 是序列化方式存储。
- **副本数量**：大于 1 时需要远程冗余备份到其他节点。如 DISK_ONLY_2 需要远程备份 1 个副本。

1) RDD 的缓存过程

RDD 在缓存到存储内存之前，Partition 中的数据一般以迭代器（Iterator）的数据结构来访问，这是 Scala 语言中一种遍历数据集合的方法。通过 Iterator 可以获取分区中每一条序列化或者非序列化的数据项(Record)，这些 Record 的对象实例在逻辑上占用了 JVM 堆内内存的 other 部分的空间，同一 Partition 的不同 Record 的存储空间并不连续。

RDD 在缓存到存储内存之后，Partition 被转换成 Block，Record 在堆内或堆外存储内存中占用一块连续的空间。将 Partition 由不连续的存储空间转换为连续存储空间的过程，Spark 称之为"展开"（Unroll）。

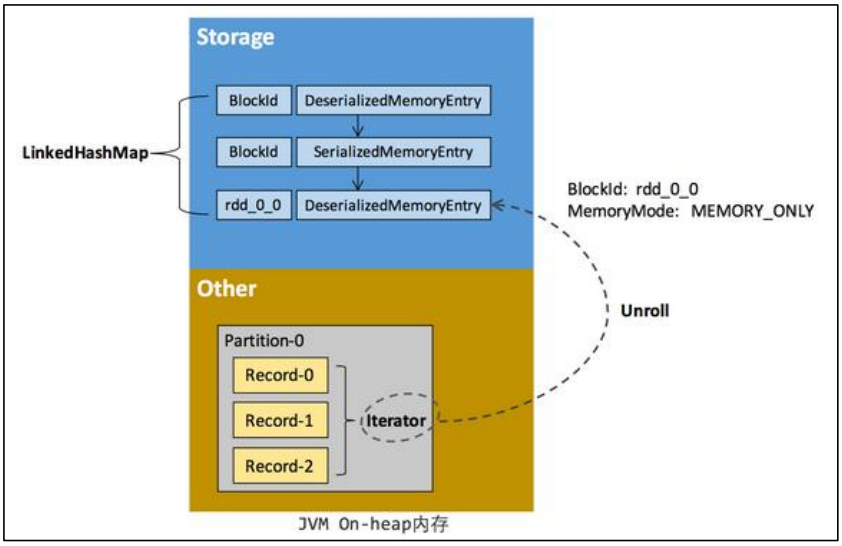
Block 有序列化和非序列化两种存储格式，具体以哪种方式取决于该 RDD 的存储级别。非序列化的 Block 以一种 DeserializedMemoryEntry 的数据结构定义，用一个数组存储所有的对象实例，序列化的 Block 则以 SerializedMemoryEntry 的数据结构定义，用字节缓冲区（ByteBuffer）来存储二进制数据。每个 Executor 的 Storage 模块用一个链式 Map 结构（LinkedHashMap）来管理堆内和堆外存储内存中所有的 Block 对象的实例，对这个 LinkedHashMap 新增和删除间接记录了内存的申请和释放。

因为不能保证存储空间可以一次容纳 Iterator 中的所有数据，当前的计算任务在 Unroll 时要向 MemoryManager 申请足够的 Unroll 空间来临时占位，空间不足则 Unroll 失败，空间足够时可以继续进行。

对于序列化的 Partition，其所需的 Unroll 空间可以直接累加计算，一次申请。

对于非序列化的 Partition 则要在遍历 Record 的过程中依次申请，即每读取一条 Record，采样估算其所需的 Unroll 空间并进行申请，空间不足时可以中断，释放已占用的 Unroll 空间。

如果最终 Unroll 成功，当前 Partition 所占用的 Unroll 空间被转换为正常的缓存 RDD 的存储空间，如下图所示。



在静态内存管理时，Spark 在存储内存中专门划分了一块 Unroll 空间，其大小是固定的，统一内存管理时则没有对 Unroll 空间进行特别区分，当存储空间不足时会根据动态占用机制进行处理。

4.3.2 淘汰与落盘

由于同一个 Executor 的所有的计算任务共享有限的存储内存空间，当有新的 Block 需要缓存但是剩余空间不足且无法动态占用时，就要对 LinkedHashMap 中的旧 Block 进行淘汰（Eviction），而被淘汰的 Block 如果其存储级别中同时包含存储到磁盘的要求，则要对其进行落盘（Drop），否则直接删除该 Block。

存储内存的淘汰规则为：

- 被淘汰的旧 Block 要与新 Block 的 MemoryMode 相同，即同属于堆外或堆内内存；
- 新旧 Block 不能属于同一个 RDD，避免循环淘汰；

- 旧 Block 所属 RDD 不能处于被读状态，避免引发一致性问题；
- 遍历 LinkedHashMap 中 Block，按照最近最少使用（LRU）的顺序淘汰，直到满足新 Block 所需的空间。其中 LRU 是 LinkedHashMap 的特性。

落盘的流程则比较简单，如果其存储级别符合_useDisk 为 true 的条件，再根据其_deserialized 判断是否是非序列化的形式，若是则对其进行序列化，最后将数据存储到磁盘，在 Storage 模块中更新其信息。

4.4 执行内存管理

执行内存主要用来存储任务在执行 Shuffle 时占用的内存，Shuffle 是按照一定规则对 RDD 数据重新分区的过程，我们来看 Shuffle 的 Write 和 Read 两阶段对执行内存的使用：

1）Shuffle Write

若在 map 端选择普通的排序方式，会采用 ExternalSorter 进行外排，在内存中存储数据时主要占用堆内执行空间。

若在 map 端选择 Tungsten 的排序方式，则采用 ShuffleExternalSorter 直接对以序列化形式存储的数据排序，在内存中存储数据时可以占用堆外或堆内执行空间，取决于用户是否开启了堆外内存以及堆外执行内存是否足够。

2）Shuffle Read

在对 reduce 端的数据进行聚合时，要将数据交给 Aggregator 处理，在内存中存储数据时占用堆内执行空间。

如果需要进行最终结果排序，则要将再次将数据交给 ExternalSorter 处理，占用堆内执行空间。

在 ExternalSorter 和 Aggregator 中，Spark 会使用一种叫 AppendOnlyMap 的哈希表在堆内执行内存中存储数据，但在 Shuffle 过程中所有数据并不能都保存到该哈希表中，当这个哈希表占用的内存会进行周期性地采样估算，当其大到一定程度，无法再从 MemoryManager 申请到新的执行内存时，Spark 就会将其全部内容存储到磁盘文件中，这个过程被称为溢存(Spill)，溢存到磁盘的文件最后会被归并(Merge)。

Shuffle Write 阶段中用到的 Tungsten 是 Databricks 公司提出的对 Spark 优化内存和 CPU 使用的计划（钨丝计划），解决了一些 JVM 在性能上的限制和弊端。Spark 会根据 Shuffle 的情况来自动选择是否采用 Tungsten 排序。

Tungsten 采用的页式内存管理机制建立在 MemoryManager 之上，即 Tungsten 对执行内存的使用进行了一步的抽象，这样在 Shuffle 过程中无需关心数据具体存储在堆内还是堆外。

每个内存页用一个 MemoryBlock 来定义，并用 Object obj 和 long offset 这两个变量统一标识一个内存页在系统内存中的地址。

堆内的 MemoryBlock 是以 long 型数组的形式分配的内存，其 obj 的值为是这个数组的对象引用，offset 是 long 型数组的在 JVM 中的初始偏移地址，两者配合使用可以定位这个数组在堆内的绝对地址；堆外的 MemoryBlock 是直接申请到的内存块，其 obj 为 null，offset 是这个内存块在系统内存中的 64 位绝对地址。Spark 用 MemoryBlock 巧妙地将堆内和堆外内存页统一抽象封装，并用页表(pageTable)管理每个 Task 申请到的内存页。

Tungsten 页式管理下的所有内存用 64 位的逻辑地址表示，由页号和页内偏移量组成：

页号：占 13 位，唯一标识一个内存页，Spark 在申请内存页之前要先申请空闲页号。

页内偏移量：占 51 位，是在使用内存页存储数据时，数据在页内的偏移地址。

有了统一的寻址方式，Spark 可以用 64 位逻辑地址的指针定位到堆内或堆外的内存，整个 Shuffle Write 排序的过程只需要对指针进行排序，并且无需反序列化，整个过程非常高效，对于内存访问效率和 CPU 使用效率带来了明显的提升。

Spark 的存储内存和执行内存有着截然不同的管理方式：对于存储内存来说，Spark 用一个 LinkedHashMap 来集中管理所有的 Block，Block 由需要缓存的 RDD 的 Partition 转化而成；而对于执行内存，Spark 用 AppendOnlyMap 来存储 Shuffle 过程中的数据，在 Tungsten 排序中甚至抽象成为页式内存管理，开辟了全新的 JVM 内存管理机制。